



الجمهورية العربية السورية

جامعة دمشق

كلية الهندسة المعلوماتية

تقرير مشروع البرمجة المتوازية 2026

تنفيذ الطلاب

سليمان مظهر يوسف

نايا خليل صالحه

علاء الدين الأمين

نور عامر شاهين

زين سمير أحمد

بإشراف المهندس :

حذيفة العقيل

١- المتطلب الأول : (Concurrent Access & Data Integrity) :

بداية للطلب الأول احتجنا لاستخدام لاراغون بدلا من سيرفر لارافيل العادي لأننا نحتاج اختبار ومعالجة طلبات متزامنة (**concurrent requests**) وقياس سلوك النظام تحت حمل حقيقي. خادم `php artisan serve` مخصص للتطوير البسيط ويعمل في عملية واحدة ولا يعكس سلوك الخادم الحقيقي عند استقبال طلبات متزامنة أو عند التعامل مع اتصالات متعددة، بينما Laragon يوفر بيئة أقرب للإنتاج (Apache/Nginx + PHP-FPM) ويسمح بتشغيل عدة عمليات متزامنة، ضبط المنافذ، وإدارة قواعد البيانات والخدمات بسهولة.

لاختبار ال `race conditation` طبقنا ذلك على عملية الاوردر حيث ان المستخدم هنا يحتاج لاختذ المنتج من المستودع بالداتا بيز و لكن في حال قام اكثر من مستخدم في نفس الوقت بالقيام بنفس العملية قد تقع ب `race conditation` و هذا ما واجهناه بالكود التالي للقيام بطلب معين :

```
return response()->json([ 'error' => 'Some cart items are invalid '], 400);
}

try {
    $order = DB::transaction(function () use ($user, $cartItems, $cart, $CartItem) {

        $productIds = $cartItems->pluck('product_id');
        $products = Product::whereIn('id', $productIds)
            ->get()
            ->keyBy('id');

        foreach ($cartItems as $item) {
            $product = $products->get($item->product_id);

            if (!$product) {
                throw new \Exception("Product not found for id {$item->product_id}");
            }

            if ($product->stock < $item->quantity) {
                throw new \App\Exceptions\InsufficientStockException(
                    "Requested quantity exceeds available stock",
                    $product->name
                );
            }
        }

        $totalPrice = $cartItems->sum(fn($i) => $i->price * $i->quantity);

        $order = Order::create([
            'user id' => $user->id,
```

لاختبار قيام اكثر من مستخدم بالطلب استخدمنا ال k6 حيث k6 يتيح تشغيل آلاف الطلبات متزامنة بطريقة قابلة للبرمجة، ما يكشف عن Race Conditions و مشاكل قفل قواعد البيانات .

```
import http from 'k6/http';
import { check } from 'k6';

export const options = {
  vus: 20,
  iterations: 20,
  // duration: '30s',
};

export default function () {

  const payload = JSON.stringify({
    items: [40]
  });

  const params = {
    headers: {
      'Content-Type': 'application/json',
      'Authorization': 'Bearer eyJ0eXAiOiJKV1QiLCJ...
    },
  };

  let res = http.post(
    'http://ppp.test/api/orders/create',
    payload,
    params
  );

  // check(res, {
  //   'status exists': (r) => r.status > 0,
  // });
  console.log(res.status);
  console.log(res.body);
}
```

سكربت k6 بسيط (20 VUs × 20 iterations) لإرسال طلبات POST إلى /api/orders/create بهدف محاكاة حمل متزامن واكتشاف مشكلات تنافسية أو أخطاء في المصادقة. السكربت يرسل حمولة JSON مع توكن مصادقة ويسجل حالة الاستجابة ومحتواها. و ليكن لدينا منتج بالداتا بيز :

94	blanditiis	Suscipit sapiente dolore nam sunt. Id cumque qui eos q...	130.0	21	2026-05-16 15:55:46	20
----	------------	---	-------	----	---------------------	----

و طبقنا التيست بحيث كل مستخدم يأخذ قطعتين من المنتج تكون النتيجة بعد تطبيق الامر k6 run race_test.js :

```
INFO[0007] {"message":"Order created successfully","order_id":828,"total_price":520} source=console
INFO[0009] 201 source=console
INFO[0009] {"message":"Order created successfully","order_id":829,"total_price":520} source=console
INFO[0009] 201 source=console
INFO[0009] {"message":"Order created successfully","order_id":830,"total_price":520} source=console
INFO[0009] 201 source=console
INFO[0009] {"message":"Order created successfully","order_id":831,"total_price":520} source=console
INFO[0009] 201 source=console
INFO[0009] {"message":"Order created successfully","order_id":832,"total_price":520} source=console
INFO[0009] 201 source=console
INFO[0009] {"message":"Order created successfully","order_id":833,"total_price":520} source=console
INFO[0009] 201 source=console
INFO[0009] {"message":"Order created successfully","order_id":834,"total_price":520} source=console
INFO[0009] 201 source=console
INFO[0009] {"message":"Order created successfully","order_id":835,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":836,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":837,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":839,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":840,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":838,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":841,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":842,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":844,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":843,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":846,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":845,"total_price":520} source=console
INFO[0010] 201 source=console
INFO[0010] {"message":"Order created successfully","order_id":847,"total_price":520} source=console

data_received.....: 7.5 kB 678 B/s
data_sent.....: 10 kB 902 B/s
http_req_blocked.....: avg=8.85ms min=1.83ms med=8.68ms max=21ms p(90)=12.08ms p(95)=12.78ms
http_req_connecting.....: avg=7.53ms min=1.64ms med=7.67ms max=19.75ms p(90)=10.21ms p(95)=10.71ms
http_req_duration.....: avg=11.57s min=9.64s med=10.46s max=33.86s p(90)=10.99s p(95)=12.17s
  { expected_response:true }...: avg=11.57s min=9.64s med=10.46s max=33.86s p(90)=10.99s p(95)=12.17s
http_req_failed.....: 0.00% 0 out of 20
http_req_receiving.....: avg=12.37ms min=194.92µs med=10.02ms max=31.14ms p(90)=23.4ms p(95)=28.71ms
http_req_sending.....: avg=8.66ms min=122.47µs med=3.01ms max=21.18ms p(90)=20.9ms p(95)=21.03ms
http_req_tls_handshaking.....: avg=0s min=0s med=0s max=0s p(90)=0s p(95)=0s
http_req_waiting.....: avg=11.54s min=9.59s med=10.43s max=33.83s p(90)=10.98s p(95)=12.16s
http_reqs.....: 20 1.806986/s
iteration_duration.....: avg=9.74s min=7.06s med=9.88s max=10.53s p(90)=10.44s p(95)=10.47s
iterations.....: 20 1.806986/s
vus.....: 8 min=8 max=20
vus_max.....: 20 min=20 max=20

running (00m11.1s), 00/20 VUs, 20 complete and 0 interrupted iterations
default ✓ [=====] 20 VUs 00m10.5s/10m0s 20/20 shared iters
```

مع ان النتيجة المتوقعة يجب ان تكون : فقط 10 مستخدمين يستطيعون القيام بالعملية بنجاح و لكن هنا نلاحظ ان الجميع قامو بالعملية و كيف أصبح المنتج بالداتا بيز ؟

94	blanditiis	Suscipit sapiente dolore nam sunt. Id cumque qui eos q...	130.0	-19	2026-05-16 15:55:46	2026
----	------------	---	-------	-----	---------------------	------

نلاحظ قيمة سالبة و هذا شيء غير صحيح مع ان الشرط :

```

if ($product->stock < $item->quantity) {
    throw new \App\Exceptions\InsufficientStockException(
        "Requested quantity exceeds available stock",
        $product->name
    );
}

```

كان من المفترض ان يمنع بقية المستخدمين من الطلب لكن ما حصل ان العمليات المتوازية جعلت جميع المستخدمين قادرين على القيام بطلب المنتج بنفس الوقت .

لاصلاح ذلك قمنا بالتالي :

```

try {
    $order = DB::transaction(function () use ($user, $cartItems, $cart, $cartItemIds) {

        $productIds = $cartItems->pluck('product_id');
        $products = Product::whereIn('id', $productIds)
            ->lockForUpdate()
            ->get()
            ->keyBy('id');

        foreach ($cartItems as $item) {
            $product = $products->get($item->product_id);

            if (!$product) {
                throw new \Exception("Product not found for id {$item->product_id}");
            }

            if ($product->stock < $item->quantity) {
                throw new \App\Exceptions\InsufficientStockException(
                    "Requested quantity exceeds available stock",
                    $product->name
                );
            }
        }

        $totalPrice = $cartItems->sum(fn($i) => $i->price * $i->quantity);

        $order = Order::create([
            'user_id' => $user->id,
            'total_price' => $totalPrice,
        ]);

        foreach ($cartItems as $item) {
            $product = $products->get($item->product_id);

            // sleep(2);
            $product->decrement('stock', $item->quantity);

            OrderItem::create([
                'order_id' => $order->id,
                'product_id' => $item->product_id,
            ]);
        }
    });
}

```

أي قمنا بوضع قفل على المنتج كي نمنع ال Race Condition فنلاحظ بعد تكرار التجربة على منتج اخر :

2- المتطلب الثاني : (Resource Management & Capacity Control):

في البداية، كانت معمارية النظام تعتمد على المعالجة المتزامنة التقليدية (Synchronous Execution) ، حيث كان منطق الدفع بكامله يقع مباشرة داخل الدالة `createPaymentIntent` في الـ Controller دون الاستعانة بوظائف خلفية مؤجلة.

- الآثار السلبية لهذه المعمارية تحت الضغط:

- زمن استجابة مرتفع (High Latency) كان على المستخدم انتظار انتهاء عملية الدفع كاملة (والتي تتضمن الاتصال ببوابة الدفع الخارجي، التحقق من المخزن، وتحديث الجداول الحساسة) قبل استقبال رد الـ HTTP.
- اختناق السيرفر (Resource Exhaustion) مع تدفق الطلبات المتزامنة بكثافة، احتجز كل طلب خيط معالجة (Thread) كامل داخل الأباتشي لفترة طويلة. هذا الاحتجاز أدى سريعاً إلى استهلاك كامل الطاقة الاستيعابية للسيرفر، مما تسبب في انهيار العمليات (Crashes) وظهور أخطاء خادم داخلية (Status Code 500) قبل أن يتمكن إطار العمل حتى من تسجيل الخطأ.

- استراتيجية التحسين وتوزيع الموارد عبر Laravel Horizon :

- قمنا باستخدام Laravel Horizon كلوحة تحكم ونظام إدارة متقدم للطوابير تسمح لنا بتهيئة العمال (Workers) وتوزيع العمليات المتوازية (Processes) بدقة وعزل الموارد (Resource Isolation).
- لمعالجة هذا الاختناق، قمنا بنقل المهام الثقيلة (مثل معالجة الدفع وإصدار الفواتير) خارج المسار الرئيسي للطلب وتحويلها إلى جوبات خلفية غير متزامنة و (Asynchronous Jobs) و سيتم شرح أهمية و كيفية تطبيق هذه العملية بشكل مفصل في الطلب الثالث .
- شكل التابع بعد منطق الدفع ل job منفصل :

```

public function createPaymentIntent(Request $request)
{
    $validator = Validator::make($request->all(), [
        'order_id' => 'required|exists:orders,id',
        'scenario' => 'nullable|string'
    ]);

    if ($validator->fails()) {
        return response()->json([
            'error' => $validator->errors()
        ], 422);
    }

    $order = Order::where('user_id', Auth::id())
        ->where('status', 'pending')
        ->with('orderItems')
        ->find($request->order_id);

    if (!$order) {
        return response()->json([
            'message' => 'Order not found or not payable'
        ], 404);
    }

    if ($order->status == 'paid') {
        return response()->json([
            'message' => 'Order already paid'
        ], 400);
    }

    ProcessOrder::dispatch(
        $order,
        Auth::id(),
        $request->scenario
    )->onQueue('payments');

    return response()->json([
        'message' => 'Payment is being processed in background',
    ], 202);
}

```

أما بالنسبة لإعدادات ال horizon فتركناها بالبداية كما هي بالصورة :


```
'defaults' => [  
  'supervisor-default' => [  
    'connection' => 'redis',  
    'queue' => ['default'],  
    'balance' => 'auto',  
    'autoScalingStrategy' => 'time',  
    'maxProcesses' => 10,  
    'memory' => 128,  
    'tries' => 1,  
    'timeout' => 60,  
  ],  
,  
  
  'environments' => [  
    'production' => [  
      'supervisor-default' => [  
        'maxProcesses' => 20,  
      ],  
    ],  
  
    'local' => [  
      'supervisor-default' => [  
        'maxProcesses' => 3,  
      ],  
    ],  
  ],  
,
```

و قمنا بتطبيق تجربة باستخدام ال k6 بالشكل التالي :

```

import http from 'http';
import { check } from 'k6';

export let options = {
  vus: 50,
  iterations: 50,
  // duration: '1m',
};

const BASE_URL = 'http://ppp.test';

const TOKEN = 'eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ3a2p3c3R0bG1uZD8wOGEvciBhdG8uZGlzC3QvYXQpL2F1ZG9yY2QwZW41IiwiaWF0Ij09XQ0p';

const params = {
  headers: {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'Authorization': 'Bearer ${TOKEN}',
  },
};

export default function () {
  // 1. CREATE ORDER
  let orderPayload = JSON.stringify({
    items: [1]
  });

  let orderRes = http.post(
    `${BASE_URL}/api/orders/create`,
    orderPayload,
    params
  );

  check(orderRes, {
    'order created': (r) => r.status === 201,
  });

  if (orderRes.status !== 201) {
    console.log('Order failed (Status ${orderRes.status}): ' + orderRes.body);
    return;
  }

  let orderData;
  try {
    orderData = JSON.parse(orderRes.body);
  } catch (e) {
    console.log('Failed to parse order response JSON: ' + orderRes.body);
    return;
  }

  let orderId = orderData.order_id;

  // 2. CREATE PAYMENT 300
  let paymentPayload = JSON.stringify({
    order_id: orderId,
    scenario: 'success'
  });

  let paymentRes = http.post(
    `${BASE_URL}/api/payment/create-intent`,
    paymentPayload,
    params
  );

  let isQueued = check(paymentRes, {
    'payment queued': (r) => r.status === 202,
  });

  if (isQueued) {
    console.log('Order ${orderId} queued successfully');
  } else {
    console.log('Payment failed for Order ${orderId} (Status ${paymentRes.status}): ' + paymentRes.body);
  }
}

```

هنا يقوم ال k6 بانشاء 50 مستخدم افتراضيين ليقومو بعمليات انشاء الطالب و من ثم الدفع .

حتى نقوم بعملية التجريب بالبداية قمنا باستخدام بيئة (WSL) Ubuntu لمحاكاة بيئة الإنتاج الحقيقية لان نظام ويندوز الافتراضي يواجه قيوداً في إدارة توازي عمليات PHP لذا وفرت بيئة WSL استقراراً كاملاً لتشغيل الـ Workers ونظام Horizon برمجياً .

عند تشغيل الامر `k6 run resource_test.js` يظهر لنا التالي :

Main		I/O																								
PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command															
9103	mysql	20	0	2823M	467M	38692	S	0.7	12.3	0:05.60	/usr/sbin/mysqld															
9106	mysql	20	0	2823M	467M	38692	S	0.8	12.3	0:05.03	/usr/sbin/mysqld															
9724	nour	20	0	202M	70536	26004	S	0.0	1.8	0:17.33	/usr/bin/php8.3 artisan horizon:work	redis	--name=default	--superv												
9725	nour	20	0	202M	70536	26008	S	0.0	1.8	0:14.43	/usr/bin/php8.3 artisan horizon:work	redis	--name=default	--superv												
9726	nour	20	0	202M	70628	26124	S	0.7	1.8	0:15.18	/usr/bin/php8.3 artisan horizon:work	redis	--name=default	--superv												
11001	root	20	0	3188	1124	980	S	0.0	0.0	0:00.00	/init															
11003	root	20	0	3188	1256	1108	S	0.0	0.0	0:00.50	/init															
11006	nour	20	0	6068	5388	3644	S	0.0	0.1	0:00.71	-bash															
13137	mysql	20	0	2823M	467M	38692	S	0.0	12.3	0:02.68	/usr/sbin/mysqld															
13437	mysql	20	0	2823M	467M	38692	S	0.0	12.3	0:00.80	/usr/sbin/mysqld															
13440	mysql	20	0	2823M	467M	38692	S	0.0	12.3	0:01.07	/usr/sbin/mysqld															

نلاحظ ان :

•الازدحام في طابور واحد :يظهر الحقل (Queue: default) أن هذه المهمة الحساسة تم رميها في الطابور الافتراضي المشترك، مما يعني أنها كانت تتزاحم مع كافة مهام النظام الأخرى دون أي أولوية.

•دليل الاختناق الزمني (الاستنتاج الأهم) :نلاحظ أن المهمة دخلت الطابور (Pushed) في الساعة 20:34:01، ولكنها لم تكتمل (Completed) إلا في الساعة 20:34:23 أي أن معالجة هذا الطلب استغرقت 22 ثانية كاملة، هذا التأخير الكبير هو دليل قاطع على حدوث اختناق الطوابير (Queue Congestion) فالعامل (Worker) كان مشغولاً بمهام سابقة متراكمة، مما أجبر هذا الطلب على الانتظار طويلاً قبل أن يحصل على دوره في التنفيذ.

لذلك انتقلنا إلى بنية المشرفين المتعددين (Multi-Supervisor) ، عبر فصل المهام واستحداث مشرف مستقل ومخصص للعمليات الحساسة باسم (supervisor-payments)

```

'defaults' => [
  'supervisor-default' => [
    'connection' => 'redis',
    'queue' => ['default'],
    'balance' => 'auto',
    'autoScalingStrategy' => 'time',
    'maxProcesses' => 10,
    'memory' => 128,
    'tries' => 1,
    'timeout' => 60,
  ],
  'supervisor-payments' => [
    'connection' => 'redis',
    'queue' => ['payments'],
    'balance' => 'simple',
    'processes' => 5,
    'maxProcesses' => 5,
    'memory' => 128,
    'tries' => 3,
    'timeout' => 90,
  ],
],

'environments' => [
  'production' => [
    'supervisor-default' => [
      'maxProcesses' => 20,
    ],
    'supervisor-payments' => [
      'processes' => 5,
      'maxProcesses' => 5,
    ],
  ],
  'local' => [
    'supervisor-default' => [
      'maxProcesses' => 3,
    ],
    'supervisor-payments' => [
      'processes' => 3,
      'maxProcesses' => 3,
    ],
  ],
],

```

أسباب التغيير :

- عزل الموارد الحسابية: (Resource Isolation) منع جوبات الدفع الثقيلة من الانتظار خلف طلبات النظام العادية في طابور واحد (مما حلّ مشكلة تأخير الـ 22 ثانية للطلب 809).
- تخفيف الاختناق: حماية جداول قاعدة البيانات من الأقفال المتلاحقة (Database Locks) عبر فصل مسارات المعالجة.

الفوائد الهندسية المحققة :

- استراتيجية موازنة ذكية: الطابور العادي يتوسع ديناميكياً حسب الحاجة ($\text{balance} \Rightarrow \text{auto}$).
- طابور المدفوعات محجوز له 3 عمال مستقرين ومستعدين دائماً محلياً ($\text{processes} = 3$) لمعالجة الدفع فوراً دون أي تأخير ميكروي.

- رفع متانة النظام عند الفشل (Fault Tolerance)
 - الجوبات العادية تحاول مرة واحدة فقط (`tries => 1`) لمنع هدر الموارد.
 - جوبات الدفع الحساسة مُنحت 3 محاولات (`tries => 3`) ومهلة أكبر 90 ثانية (`timeout => 90`) لضمان عدم ضياع عملية الدفع في حال حدوث مشاكل في شبكة البوابة الخارجية.
- التحكم بالطاقة الاستيعابية حسب البيئة (Capacity Control):
 - في الـ `local` حصر العمليات بـ 3 عمال لتناسب موارد الجهاز المحلي.
 - في الـ `production` رفع السقف تلقائياً إلى 20 عاملاً للـ `default` و 5 عمال للـ `payments` لمنع أي طفرة طلبات متزامنة في السيرفر الحقيقي.

Main										
PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+ Command
508	root	20	0	6072	5300	3616	S	0.0	0.1	0:00.02 -bash
577	mysql	20	0	2823M	466M	38692	S	0.0	12.3	0:00.93 /usr/sbin/mysqld
1004	root	20	0	3188	1124	980	S	0.0	0.0	0:00.00 /init
1005	root	20	0	3188	1128	980	S	0.0	0.0	0:00.25 /init
1012	root	20	0	6068	5384	3644	S	0.0	0.1	0:00.27 -bash
1742	root	20	0	3188	1124	980	S	0.0	0.0	0:00.00 /init
1744	root	20	0	3188	1256	1108	S	0.0	0.0	0:01.65 /init
1747	root	20	0	6196	5424	3648	S	0.0	0.1	0:00.59 -bash
9100	mysql	20	0	2823M	466M	38692	S	0.0	12.3	0:02.93 /usr/sbin/mysqld
9102	mysql	20	0	2823M	466M	38692	S	0.0	12.3	0:02.25 /usr/sbin/mysqld
9709	root	20	0	122M	50696	23480	S	0.0	1.5	0:04.27 /usr/bin/php8.3 artisan horizon:supervisor desktop-rlm643g-Hq3r:supervisor-default
9710	root	20	0	122M	50780	23528	S	0.7	1.5	0:04.35 /usr/bin/php8.3 artisan horizon:supervisor desktop-rlm643g-Hq3r:supervisor-paymen
9727	root	20	0	122M	59944	23856	S	0.0	1.5	0:04.63 /usr/bin/php8.3 artisan horizon:work redis --name=default --supervisor=desktop-rl
11001	root	20	0	3188	1124	980	S	0.0	0.0	0:00.00 /init
11003	root	20	0	3188	1256	1108	S	0.0	0.0	0:00.16 /init
11006	root	20	0	6068	5384	3644	S	0.0	0.1	0:00.24 -bash
11048	mysql	20	0	2823M	466M	38692	S	0.0	12.3	0:01.04 /usr/sbin/mysqld

- المشرف العام لقنوات النظام.(supervisor-default)
- المشرف المخصص لقنوات الدفع والعمليات الثقيلة.(supervisor-payments)

- انخفاض زمن المعالجة المتراكم وصدمة الاستجابة: (TIME+) رُصد هبوط حاد وسريع في مؤشر وقت المعالجة الحسابي الكلي للـ Workers ليتراجع من 17 ثانية ليصل إلى 4 ثوان فقط. (0:04.35)

نرى ثبات مؤشر استهلاك المعالج اللحظي (CPU%) عند قيم صفرية في الحالتين لأن جوبات المعالجة تعتمد أساساً على كفاءة الإدخال والإخراج (I/O-Bound) والانتظار الشبكي لبوابات الدفع وقاعدة البيانات، وليس على العمليات الحسابية

المعقدة (CPU-Bound). ولذلك، فإن المقياس الهندسي الأدق المعبر عن تحسن الأداء في الفحص هو تراجع الوقت المتراكم لتنفيذ العمليات (TIME+) من 17 ثانية إلى 4 ثوانٍ نتيجة عزل الطوابير

و بالرجوع لل horizon نلاحظ بالصورة التالية:

Completed Jobs			
Job	Queued	Completed	Runtime
ProcessOrder Queue: payments Tags: App\Models\Order:759	2026-05-18 19:18:41	2026-05-18 19:18:49	1.28s
ProcessOrder Queue: payments Tags: App\Models\Order:757	2026-05-18 19:18:41	2026-05-18 19:18:48	1.47s
ProcessOrder Queue: payments Tags: App\Models\Order:758	2026-05-18 19:18:41	2026-05-18 19:18:48	1.25s
ProcessOrder Queue: payments Tags: App\Models\Order:756	2026-05-18 19:18:41	2026-05-18 19:18:48	1.21s
ProcessOrder Queue: payments Tags: App\Models\Order:754	2026-05-18 19:18:41	2026-05-18 19:18:47	1.20s
ProcessOrder Queue: payments Tags: App\Models\Order:755	2026-05-18 19:18:41	2026-05-18 19:18:47	1.17 

يُلاحظ من توقيت الجوبات المتتالية لـ `ProcessOrder` أنها دفعت كلها في نفس الثانية.(19 : 18 : 41)

المعالجة الفعلية لكل جوب (Runtime) استغرقت ثانية وبضع أجزاء فقط تتراوح بين 1.17 s إلى 1.47 s واكتملت فوراً. هذا يثبت التخلص التام من اختناق الـ 22 ثانية السابق، حيث يعالج العمال المهام بالتوازي اللحظي بمجرد دخولها الطابور

Overview			
Jobs Per Minute	Jobs Past Hour	Failed Jobs Past 7 Days	Status
0	61	2	Inactive
Total Processes	Max Wait Time	Max Runtime	Max Throughput
4	-	payments	payments
Current Workload			
Queue	Jobs	Processes	Wait
default	0	1	A few seconds
payments	0	3	A few seconds

إجمالي العمليات المستقرة: تظهر اللوحة حجز 4 عمليات متوازية (Total Processes: 4) بشكل نشط ومستمر في الخلفية.

توزيع العمال الذكي: تم تخصيص عامل واحد (Processes: 1) لطابور المهام العادية default، بينما تم حجز 3 عمال متفرغين (Processes: 3) لطابور المدفوعات payments. هذا يتضمن تصفير زمن الانتظار (Wait: A few seconds) وجعل الموارد متأهبة دائماً لاستقبال أي ضغط متزامن.

Metrics	
Jobs	Queues
Queue	
default	
payments	

يؤكد تبويب القياسات نجاح عزل النظام برمجياً إلى قناتين مستقلتين تماماً هما default و payments مما يتيح تتبع ومراقبة مؤشرات أداء كل قطاع (Business Domain) في النظام بشكل منفصل ودقيق لضمان جودة الخدمة .

3- المتطلب الثالث : التحقق من المعالجة غير المتزامنة وأنظمة الطوابير (Asynchronous Queues):

قمنا بتطبيق هذا المتطلب أثناء توليد الفاتورة الخاصة بالمستخدم عند عملية الطلب.

تم الانتقال بالنظام من النمط المتزامن التقليدي (Synchronous Blocking) إلى النمط غير المتزامن (Asynchronous Non-blocking) باستخدام أنظمة الطوابير المتقدمة. أثبتت نتائج الاختبار العملية باستخدام K6 و Laravel Horizon تقليل زمن استجابة واجهة برمجة التطبيقات (API Response Time) من عدة ثوان إلى 89 ميلي ثانية مع ضمان معالجة المهام الثقيلة في الخلفية دون التأثير على تجربة المستخدم.

- المعمارية :

- تعتمد المعمارية الحديثة للنظام على فصل المهام (Decoupling) عبر تجزئة عملية الشراء إلى مرحلتين أساسيتين :
- ١- المسار الحرج : جلب الطلب , والتحقق من حالته , وإطلاق مهمة الدفع الخلفية ثم إعادة استجابة فورية للمستخدم برقم الحالة 202.
 - ٢- المسار الغير متزامن: يلتقط خادم الطوابير المهمة و يتصل ببوابة الدفع Stripe و عند النجاح , يطلق مهمة فرعية مستقلة تماماً لتوليد الفاتورة بصيغة PDF وحفظها .

- منهجية و جاهزية الاختبار :

لإثبات نجاح الفصل المعماري تم اعتماد استراتيجيات Heavy Load Simulation وفق الخطوات التالية :

```

class GenerateInvoicePDF implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    protected $order;

    public function __construct(Order $order)
    {
        $this->order = $order;
    }

    public function handle()
    {
        logger()->info("[PDF Job] Started generating invoice for Order #{$this->order->id}");

        sleep(5);

        $order = $this->order->load(['orderItems.product', 'user']);

        $data = [
            'title' => ' invoice number ' . $this->order->id,
            'id' => $order->id,
            'date' => $order->created_at->format('d/m/Y'),
            'name' => $order->user->name,
            'items' => $order->orderItems,
            'total' => $order->total_price,
        ];

        $pdf = Pdf::loadView('invoices.order_pdf', $data);
        $path = 'invoices/invoice_' . $order->id . '.pdf';
        if (!Storage::exists('public/invoices')) {
            Storage::makeDirectory('public/invoices');
        }

        Storage::disk('public')->put('invoices/invoice_' . $this->order->id . '.pdf', $pdf->output());
        $order->invoice_path = $path;
        $order->save();

        $this->order->update([
            'processed_by' => $this->queue
        ]);
        logger()->info("PDF Generated for Order #{$order->id}");
    }
}

```

- تفعيل الدالة sleep(5) ضمن الوظيفة الخاصة بتوليد الفاتورة (GenerateInvoicePDF) و اجبارها على استهلاك 5 ثوان كاملة تحاكي برمجيا معالجة ملفات PDF ضخمة ومعقدة تحت البيئة التي نحاكها.
- تم تصميم script خاص باستخدام K6 لضخ طلبات متوازية و متزامنة في نفس الجزء من الثانية عبر مستخدمين وهميين (VUs).

اختبار استجابة واجهة التطبيقات الأمامية (Client-Side Performance Test)
 تم تشغيل اختبار k6 لإرسال طلبات دفع متوازية لعدة طلبات معلقة في قاعدة البيانات للتأكد من سلوك الـ API.

```

  V K C

execution: local
script: async_test.js
output: -

scenarios: (100.00%) 1 scenario, 3 max VUs, 10m30s max duration (incl. graceful stop):
* default: 3 iterations shared among 3 VUs (maxDuration: 10m0s, gracefulStop: 30s)

INFO[0000] [k6] API Response for Order 1004: 202 | Time: 89ms source=console
INFO[0000] [k6] API Response for Order 1002: 202 | Time: 89ms source=console
INFO[0000] [k6] API Response for Order 1003: 202 | Time: 89ms source=console

■ TOTAL RESULTS

checks_total.....: 6 66.257919/s
checks_succeeded...: 100.00% 6 out of 6
checks_failed.....: 0.00% 0 out of 6

✓ is status 202 (Accepted)
✓ response is fast (< 500ms)

HTTP
http_req_duration.....: avg=89.2ms min=88.98ms med=89.3ms max=89.3ms p(90)=89.3ms p(95)=89.3ms
{ expected_response:true }...: avg=89.2ms min=88.98ms med=89.3ms max=89.3ms p(90)=89.3ms p(95)=89.3ms
http_req_failed.....: 0.00% 0 out of 3
http_reqs.....: 3 33.12896/s

EXECUTION
iteration_duration.....: avg=90.28ms min=90.22ms med=90.28ms max=90.34ms p(90)=90.33ms p(95)=90.33ms
iterations.....: 3 33.12896/s

NETWORK
data_received.....: 972 B 11 kB/s
data_sent.....: 1.6 kB 18 kB/s
```

يظهر الفحص المخبري نجاحاً مطلقاً للفحوصات بنسبة 100% (checks_succeeded: 6 out of 6). استقبل النظام الطلبات المتوازية وأعاد الرمز البروتوكولي 202 Accepted في زمن استجابة خارق قدره 89.2ms. هذا يثبت أن العميل (Client) قد تم تحريره فوراً ولم يتم إجباره على انتظار العمليات الخلفية.

تكشف سجلات Horizon الآلية الميكانيكية الحقيقية للنظام؛ حيث تظهر الوظيفة الأساسية ProcessOrder مستغرقة ثانيتين فقط لإنهاء المعاملة المادية مع Stripe ، لتقوم فوراً بوضع وظيفة الفاتورة GenerateInvoicePDF قيد التشغيل المباشر. تظهر عبارة 5s DONE متكررة بوضوح أمام وظائف الفواتير، مما يؤكد هندسياً أن الثقل الزمني البالغ 5 ثوانٍ تم عزله واستهلاكه بالكامل داخل خادم الخلفية دون أن يتأثر زمن استجابة المستخدم المقدر بـ 89ms.

```
naya@naya:~/projects/E-commerce$ php artisan horizon
```

```
INFO Horizon started successfully.
```

```
2026-05-18 14:05:02 App\Jobs\ProcessOrder ..... RUNNING
2026-05-18 14:05:02 App\Jobs\ProcessOrder ..... RUNNING
2026-05-18 14:05:04 App\Jobs\ProcessOrder ..... 2s DONE
2026-05-18 14:05:04 App\Jobs\GenerateInvoicePDF ..... RUNNING
2026-05-18 14:05:05 App\Jobs\ProcessOrder ..... 2s DONE
2026-05-18 14:05:05 App\Jobs\GenerateInvoicePDF ..... RUNNING
2026-05-18 14:05:05 App\Jobs\GenerateInvoicePDF ..... 906.20ms DONE
2026-05-18 14:05:10 App\Jobs\GenerateInvoicePDF ..... 5s DONE
2026-05-18 14:06:50 App\Jobs\ProcessOrder ..... RUNNING
2026-05-18 14:06:52 App\Jobs\ProcessOrder ..... RUNNING
2026-05-18 14:06:53 App\Jobs\ProcessOrder ..... 2s DONE
2026-05-18 14:06:53 App\Jobs\GenerateInvoicePDF ..... RUNNING
2026-05-18 14:06:54 App\Jobs\ProcessOrder ..... 1s DONE
2026-05-18 14:06:56 App\Jobs\GenerateInvoicePDF ..... RUNNING
2026-05-18 14:06:58 App\Jobs\GenerateInvoicePDF ..... 5s DONE
2026-05-18 14:07:02 App\Jobs\GenerateInvoicePDF ..... 5s DONE
2026-05-18 14:08:05 App\Jobs\ProcessOrder ..... RUNNING
2026-05-18 14:08:06 App\Jobs\ProcessOrder ..... RUNNING
2026-05-18 14:08:07 App\Jobs\ProcessOrder ..... 2s DONE
2026-05-18 14:08:07 App\Jobs\GenerateInvoicePDF ..... RUNNING
2026-05-18 14:08:09 App\Jobs\ProcessOrder ..... 3s DONE
2026-05-18 14:08:09 App\Jobs\GenerateInvoicePDF ..... RUNNING
2026-05-18 14:08:09 App\Jobs\GenerateInvoicePDF ..... 1s DONE
2026-05-18 14:08:14 App\Jobs\GenerateInvoicePDF ..... 5s DONE
```

الخطوة الأخيرة هي مطابقة ال (TimeStamps) داخل ملفات سجلات النظام المباشرة ب (Laravel.log).

```

running (00m00.1s), 0/3 VUs, 3 complete and 0 interrupted iterations
default ✓ [=====] 3 VUs 00m00.1s/10m0s 3/3 shared iters
naya@Naya:~/projects/E-commerce$ cat storage/logs/laravel.log
[2026-05-18 12:26:05] local.INFO: Analytics Job started on: default
[2026-05-18 12:26:06] local.INFO: Analytics Job finished on: default
[2026-05-18 12:26:08] local.INFO: Analytics Job started on: default
[2026-05-18 12:26:08] local.INFO: Analytics Job finished on: default
[2026-05-18 12:37:10] local.INFO: Analytics Job started on: default
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 1 | last id: 500
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 501 | last id: 1000
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 1001 | last id: 1500
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 1501 | last id: 2000
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 2001 | last id: 2500
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 2501 | last id: 3000
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 3001 | last id: 3500
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 3501 | last id: 4000
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 4001 | last id: 4500
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 4501 | last id: 5000
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 5001 | last id: 5500
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 5501 | last id: 6000
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 6001 | last id: 6500
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 6501 | last id: 7000
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 7001 | last id: 7500
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 7501 | last id: 8000
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 8001 | last id: 8500
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 8501 | last id: 9000
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 9001 | last id: 9500
[2026-05-18 12:37:11] local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 9501 | last id: 10000
[2026-05-18 12:37:12] local.INFO: Analytics Job finished on: default

```

4- المتطلب الرابع: معالجة البيانات الضخمة على دفعات (Batch Processing):

تم تطبيق هذا المتطلب أثناء عملية جرد المبيعات اليومية (إنشاء التقارير)

لإثبات كفاءة النظام، تم توليد عينة بيانات ضخمة (Data Seeding) تحاكي بيئة الإنتاج الحقيقية.
التقنية المستخدمة: تم استخدام `DB::table()->insert()` مع تعطيل قيود المفاتيح الأجنبية
`Schema::disableForeignKeyConstraints()` مؤقتاً لتسريع عملية حقن
 السجلات في جدول `orders`.
حجم العينة (10,000): لتاريخ الأمس

- **المعمارية:** تم إنشاء فئة `DailySalesAnalyticsJob` وتطبيق واجهة `ShouldQueue` لإجبار إطار العمل على تحويلها إلى طابور المعالجة (Queue) بدلاً من التنفيذ المتزامن.
 كود المعالجة الأساسي (تم تطبيق نمط الدفعات):


```

Order::chunkById(500, function ($orders) use (
    &$totalOrders,
    &$totalSales,
    &$paidOrders,
    &$canceledOrders
) {

    logger()->info("=== [Chunk Detected] === number of chunks in batch " . $orders->count() . "

    foreach ($orders as $order) {
        $totalOrders++;
        $totalSales += $order->total_price;

        if ($order->status === 'paid') {
            $paidOrders++;
        }

        if ($order->status === 'canceled') {
            $canceledOrders++;
        }
    }
});

```

- منهجية وجاهزية الاختبار :

- تم تكوين النظام لاستخدام محرك Redis عبر مكتبة (predis) ونظام Horizon لإدارة الطوابير. عند إطلاق الوظيفة، لم يتوقف النظام لانتظار النتيجة، بل تم إرسالها فوراً إلى الخلفية.

```
naya@Naya:~/projects/E-commerce$ php artisan horizon
```

```
INFO Horizon started successfully.
```

```

2026-05-18 12:26:08 App\Jobs\DailySalesAnalyticsJob ..... RUNNING
2026-05-18 12:26:08 App\Jobs\DailySalesAnalyticsJob ..... 406.33ms DONE
2026-05-18 12:26:20 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 12:26:20 Laravel\Telescope\Jobs\ProcessPendingUpdates 41.89ms DONE
2026-05-18 12:26:32 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 12:26:32 Laravel\Telescope\Jobs\ProcessPendingUpdates 16.97ms DONE
2026-05-18 12:26:44 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 12:26:44 Laravel\Telescope\Jobs\ProcessPendingUpdates 28.20ms DONE

```

^C

منع انهيار خادم PHP تحت ضغط البيانات، تم استخدام دالة (chunkById(500) تم توثيق التقسيم برمجياً من خلال طباعة معرفات البداية والنهاية لكل دفعة.

النتيجة الفنية: أثبت السجل أن النظام قام بسحب 500 سجل فقط في كل دورة، وانتقل بسلسلة من المعرف (ID) رقم 4001 إلى 4500، ثم من 4501 إلى 5000، وهكذا.

```

aya@Naya:~/projects/E-commerce$ tail -n 20 storage/logs/laravel.log
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 501 | last id: 1000
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 1001 | last id: 1500
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 1501 | last id: 2000
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 2001 | last id: 2500
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 2501 | last id: 3000
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 3001 | last id: 3500
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 3501 | last id: 4000
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 4001 | last id: 4500
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 4501 | last id: 5000
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 5001 | last id: 5500
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 5501 | last id: 6000
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 6001 | last id: 6500
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 6501 | last id: 7000
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 7001 | last id: 7500
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 7501 | last id: 8000
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 8001 | last id: 8500
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 8501 | last id: 9000
2026-05-18 12:37:11 local.INFO: === [Chunk Detected] === number of chunks in batch 500 | first id : 9001 | last id: 9500
2026-05-18 12:37:12 local.INFO: Analytics Job finished on: default
aya@Naya:~/projects/E-commerce$

```

تم التحقق بنجاح من أن النظام قادر على أتمتة تقارير المبيعات اليومية بشكل موثوق. تعمل الوظيفة بشكل خفي (Asynchronous) ، تحمي الخادم من الانهيار (Chunking) ، وتولد تقريراً دقيقاً يُحفظ في جدول `daily_sales_reports` بكفاءة عالية. (High Performance)

5- المتطلب الخامس والأخير : (Load Balancing)

تم استخدام استراتيجية توزيع تعتمد على modulo hashing لتوجيه الطلبات إلى عدة طوابير تمثل خوادم مستقلة افتراضياً، بهدف محاكاة توزيع الحمل وتحسين استقرار النظام تحت الضغط

يحدد رقم الطلب الـ queue المستهدفة باستخدام المعادلة:

$$queue = (order_id \bmod 3) + 1$$

```

// distribute jobs between workers
$workerId = ($order->id % 3) + 1;
$targetQueue = "server_" . $workerId;

ProcessOrder::dispatch(
    $order,
    Auth::id(),
    $request->scenario
)->onQueue($targetQueue);

```


ال Load Balancing هو استقبال تدفق كبير من البيانات أو الطلبات في نفس اللحظة، وتوزيعها بذكاء وهنا استخدمت خوارزمية التناوب Round Robin على مجموعة من الموارد (الخوادم) لضمان عدم اختناق سيرفر واحد، مما يضمن بقاء النظام سريعاً ومستقراً تحت الضغط.

قمنا بتعديل horizon و إضافة supervisor_servers من أجل محاكاة عمل 3 سيرفرات وتوزيع الحمل عليها :

```
'supervisor-server-1' => [  
  'connection' => 'redis',  
  'queue' => ['server_1'],  
  'balance' => 'simple',  
  'processes' => 2,  
  'tries' => 3,  
],  
  
'supervisor-server-2' => [  
  'connection' => 'redis',  
  'queue' => ['server_2'],  
  'balance' => 'simple',  
  'processes' => 2,  
  'tries' => 3,  
],  
  
'supervisor-server-3' => [  
  'connection' => 'redis',  
  'queue' => ['server_3'],  
  'balance' => 'simple',  
  'processes' => 2,  
  'tries' => 3,  
],  
],
```

هذه الصور أيضا تظهر النتائج :

```
naya@Naya: ~/projects/E-cor × + v
naya@Naya:/mnt/c/WINDOWS/system32$ cd ~/projects/E-commerce
naya@Naya:~/projects/E-commerce$ tail -n 20 storage/logs/laravel.log
[2026-05-18 14:34:36] local.INFO: [PDF Job] Started generating invoice for Order #1004
[2026-05-18 14:34:36] local.INFO: Order #1003: Payment processed by . Invoice sent to queue.
[2026-05-18 14:34:36] local.INFO: [PDF Job] Started generating invoice for Order #1003
[2026-05-18 14:34:37] local.INFO: PDF Generated for Order #1003
[2026-05-18 14:34:39] local.INFO: Order #1002: Payment processed by . Invoice sent to queue.
[2026-05-18 14:34:39] local.INFO: [PDF Job] Started generating invoice for Order #1002
[2026-05-18 14:34:40] local.INFO: PDF Generated for Order #1004
[2026-05-18 14:34:44] local.INFO: PDF Generated for Order #1002
[2026-05-18 19:14:59] local.INFO: Background Processing: Order #3 is being handled by Worker: server_1
[2026-05-18 19:14:59] local.INFO: Background Processing: Order #2 is being handled by Worker: server_3
[2026-05-18 19:14:59] local.INFO: Background Processing: Order #1 is being handled by Worker: server_2
[2026-05-18 19:15:01] local.INFO: Order #2: Payment processed by server_3. Invoice sent to queue.
[2026-05-18 19:15:01] local.INFO: Order #3: Payment processed by server_1. Invoice sent to queue.
[2026-05-18 19:15:02] local.INFO: Order #1: Payment processed by server_2. Invoice sent to queue.
[2026-05-18 19:15:02] local.INFO: [PDF Job] Started generating invoice for Order #2
[2026-05-18 19:15:02] local.INFO: [PDF Job] Started generating invoice for Order #3
[2026-05-18 19:15:05] local.INFO: [PDF Job] Started generating invoice for Order #1
[2026-05-18 19:15:07] local.INFO: PDF Generated for Order #3
[2026-05-18 19:15:07] local.INFO: PDF Generated for Order #2
[2026-05-18 19:15:10] local.INFO: PDF Generated for Order #1
naya@Naya:~/projects/E-commerce$ |
```

هذه الصورة توضح بدقة تشغيل الثلاث سيرفرات في نفس الوقت مما يدل على ان الحمل فعلا يتوزع عليها .

هذا يثبت مبدأ فصل الاهتمامات (Separation of Concerns) بمجرد اكتمال الدفع، لم يقم النظام بتوليد الفاتورة فوراً ليجعل العميل ينتظر، بل أرسل "أمر توليد الفاتورة" إلى طابور جديد وانتهت مهمة الدفع. هذا يضمن سرعة الاستجابة للمستخدم (Response Time) بأجزاء من الثانية.

عملية إنشاء ملفات الـ PDF معقدة وتستهلك موارد المعالج (CPU) السجلات توضح أن معالجة فاتورة واحدة استغرقت حوالي 5 ثوانٍ. لو كانت هذه العملية متزامنة (Synchronous) ، لتجمدت شاشة المستخدم لمدة 5 ثوانٍ لكل عملية شراء. لكن نقلها كـ Background Job (تنفيذاً للبند الرابع) جعلها تُعالج بهدوء في الخلفية دون أي تأثير على أداء خادم الويب الأساسي (Nginx) أو واجهة المستخدم.

```
naya@Naya:~/projects/E-commerce$ sudo service redis-server start
[sudo: authenticate] Password:
naya@Naya:~/projects/E-commerce$ php artisan horizon

INFO Horizon started successfully.

2026-05-18 19:14:59 App\Jobs\ProcessOrder ..... RUNNING
2026-05-18 19:14:59 App\Jobs\ProcessOrder ..... RUNNING
2026-05-18 19:14:59 App\Jobs\ProcessOrder ..... RUNNING
2026-05-18 19:15:01 App\Jobs\ProcessOrder ..... 2s DONE
2026-05-18 19:15:01 App\Jobs\ProcessOrder ..... 2s DONE
2026-05-18 19:15:02 App\Jobs\GenerateInvoicePDF ..... RUNNING
2026-05-18 19:15:02 App\Jobs\GenerateInvoicePDF ..... RUNNING
2026-05-18 19:15:02 App\Jobs\ProcessOrder ..... 2s DONE
2026-05-18 19:15:05 App\Jobs\GenerateInvoicePDF ..... RUNNING
2026-05-18 19:15:07 App\Jobs\GenerateInvoicePDF ..... 5s DONE
2026-05-18 19:15:07 App\Jobs\GenerateInvoicePDF ..... 5s DONE
2026-05-18 19:15:10 App\Jobs\GenerateInvoicePDF ..... 5s DONE
2026-05-18 19:15:13 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 19:15:13 Laravel\Telescope\Jobs\ProcessPendingUpdates ... 16.13ms DONE
2026-05-18 19:15:13 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 19:15:13 Laravel\Telescope\Jobs\ProcessPendingUpdates ... 7.26ms DONE
2026-05-18 19:15:13 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 19:15:13 Laravel\Telescope\Jobs\ProcessPendingUpdates ... 7.24ms DONE
2026-05-18 19:15:25 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 19:15:25 Laravel\Telescope\Jobs\ProcessPendingUpdates ... 15.36ms DONE
2026-05-18 19:15:25 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 19:15:25 Laravel\Telescope\Jobs\ProcessPendingUpdates ... 7.47ms DONE
2026-05-18 19:15:25 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 19:15:25 Laravel\Telescope\Jobs\ProcessPendingUpdates ... 7.09ms DONE
2026-05-18 19:15:37 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 19:15:37 Laravel\Telescope\Jobs\ProcessPendingUpdates ... 8.41ms DONE
2026-05-18 19:15:37 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 19:15:37 Laravel\Telescope\Jobs\ProcessPendingUpdates ... 9.19ms DONE
2026-05-18 19:15:37 Laravel\Telescope\Jobs\ProcessPendingUpdates ... RUNNING
2026-05-18 19:15:37 Laravel\Telescope\Jobs\ProcessPendingUpdates ... 10.16ms DONE
```

هنا ال loggers تظهر أيضا اثبات توزيع الحمل بالتوازي في نفس الثانية (19:14:59) ، دخلت 3 وظائف دفع (Payment Jobs) مختلفة إلى النظام. لو كان النظام يعتمد على خادم معالجة واحد (Single Worker) ، لتم تنفيذها بالتوالي (واحدة تلو الأخرى) ، مما يسبب تأخيراً للمستخدمين. ولكن بسبب توزيع الأحمال الذي قمنا ببنائه عبر خوارزمية التناوب وتحجيم المشرفين (supervisor-server-1, 2, 3) في خادم Horizon ، تم التقاط الوظائف الثلاث في نفس الأجزاء من الثانية ومعالجتها بالتوازي (Parallel Execution) ، وهذا هو صلب مفهوم الـ High Availability وتوزيع الأحمال."

ملاحظة:

- ١- أثناء العمل وجدنا أنه الطلبات متقاطعة مع بعضها بشكل كبير ما أدى إلى تقاطع في المعالجة وتعقيد في الفصل بين الحالات.
- ٢- هذا التقرير قام أكثر من عضو في الغروب التعديل عليه لذلك من الممكن أن يكون هناك اختلاف في طريقة الكتابة.

6- المتطلب السادس (Distributed Caching) :

المنتجات الأكثر طلباً :

ليكن لدينا تابع لعرض المنتجات الأكثر طلباً داخل ال ProductController بالشكل التالي :

```
1 public function showTopselling()
2 {
3     try {
4         $topProducts = Product::select('id', 'name', 'description', 'price')
5             ->orderBy('order_count', 'desc')
6             ->take(10)
7             ->get();
8
9         return response()->json([
10             'success' => true,
11             'message' => 'Top selling products retrieved successfully',
12             'data'    => $topProducts
13         ], 200);
14     } catch (\Exception $e) {
15         return response()->json([
16             'success' => false,
17             'error'    => 'Failed to retrieve top selling products',
18             'details'  => $e->getMessage()
19         ], 500);
20     }
21 }
```

حيث يقوم التابع بعرض أكثر 10 منتجات طلباً بشكل تنازلي بناء على كمية الطلبات لكل منتج .

و لنقم بالتجربة التالية بواسطة ال k6 (topselling.js) :

```
1 import http from 'k6/http';
2 import { check, sleep } from 'k6';
3
4 export const options = {
5   vus: 1000,
6   duration: '30s',
7 };
8
9 const BASE_URL = 'http://ppp.test/api/showTopselling';
10
11 export default function () {
12   const params = {
13     headers: {
14       'Content-Type': 'application/json',
15       'Accept': 'application/json',
16     },
17   };
18
19   let res = http.get(BASE_URL, params);
20
21   check(res, {
22     'status is 200': (r) => {
23       if (r.status !== 200) console.log(`Failed with status: ${r.status}`);
24       return r.status === 200;
25     },
26   });
27
28   sleep(0.1);
29 }
```

هنا قمنا بالتجربة على 1000 مستخدم لمدة 30 ثانية على التوازي باستخدام nginx على ubuntu
و تكون النتيجة بالشكل :

TOTAL RESULTS

```
checks_total.....: 5186      1120.469137/s
checks_succeeded...: 100.00% 5186 out of 5186
checks_failed.....: 0.00%   0 out of 5186
```

✓ status is 200

HTTP

```
http_req_duration.....: avg=8.21s min=39.44µs med=6.49s max=39.86s p(90)=38.39s p(95)=39.54s
{ expected_response:true }...: avg=8.21s min=39.44µs med=6.49s max=39.86s p(90)=38.39s p(95)=39.54s
http_req_failed.....: 0.00%   0 out of 5186
http_reqs.....: 5186      1120.469137/s
```

EXECUTION

```
iteration_duration.....: avg=6.4s  min=193.02ms med=7.1s  max=8.42s  p(90)=7.97s p(95)=8.15s
iterations.....: 5186      1120.469137/s
vus.....: 147      min=147      max=1000
vus_max.....: 1000    min=1000    max=1000
```

NETWORK

```
data_received.....: 12 MB  2.7 MB/s
data_sent.....: 726 kB 157 kB/s
```

```
running (0m04.6s), 0000/1000 VUs, 5186 complete and 0 interrupted iterations
default ✓ [=====] 1000 VUs  30s
```

تحليل النتيجة :

- استقرار كامل وبدون انهيار :النظام نجح في معالجة 5186 طلباً بالكامل، نسبة النجاح بلغت 100% دون حدوث أي خطأ في السيرفر أو ضياع للبيانات.
- تحليل مشكلة البطء والتعليق :رغم عدم انهيار السيرفر، إلا أن المستخدمين واجهوا بطئاً كبيراً؛ حيث كان متوسط زمن الاستجابة 8.21 ثانية، وارتفع في حده الأقصى إلى 39.86 ثانية للطلبات الأخيرة التي دخلت النظام.
- التفسير :هذا البطء طبيعي لأن النظام يعمل حالياً على سيرفر واحد فقط وبدون موزع أحمال؛ مما أجبر النظام على وضع الطلبات في طابور انتظار ليعالجها واحداً تلو الآخر بالترتيب .

و عند التجربة على 1500 مستخدم نلاحظ النتيجة التالية :

```

TOTAL RESULTS

checks_total.....: 7093 188.004765/s
checks_succeeded...: 78.54% 5571 out of 7093
checks_failed.....: 21.45% 1522 out of 7093

X status is 200
  78% - ✓ 5571 / X 1522

HTTP
http_req_duration.....: avg=8.65s min=0s med=8.37s max=43.99s p(90)=12.92s p(95)=40.89s
{ expected_response:true }...: avg=11.01s min=57.21µs med=9.37s max=43.99s p(90)=12.99s p(95)=41.02s
http_req_failed.....: 21.45% 1522 out of 7093
http_reqs.....: 7093 188.004765/s

EXECUTION
iteration_duration.....: avg=7.15s min=100.34ms med=8.68s max=13.17s p(90)=12.51s p(95)=12.83s
iterations.....: 7093 188.004765/s
vus.....: 7 min=7 max=1500
vus_max.....: 1500 min=1500 max=1500

NETWORK
data_received.....: 13 MB 352 kB/s
data_sent.....: 921 kB 24 kB/s

running (0m37.7s), 0000/1500 VUs, 7093 complete and 0 interrupted iterations
default ✓ [=====] 1500 VUs 30s
```

تحليل النتيجة :

- حدوث أخطاء وفشل في الطلبات :على عكس تجربة الـ 1000 مستخدم (التي كانت بطيئة ولكن نجحت بالكامل)، هنا النظام لم يتحمل؛ فمن بين 7093 طلباً إجمالياً، فشل 1522 طلباً بنسبة خطأ بلغت 21.45%. هذا يعني أن الخادم بدأ يرفض ربع طلبات المستخدمين تقريباً بسبب الحمل الزائد.
- أزمنة انتظار عالية :بقي متوسط زمن الاستجابة مرتفعاً جداً عند 8.65 ثانية، بينما انتظر المستخدم الأخير في الطابور مدة 43.99 ثانية كاملة قبل أن يستجيب له السيرفر أو يرفضه.

و عند التجربة على 2000 مستخدم:

```

TOTAL RESULTS

checks_total.....: 38374  5786.236354/s
checks_succeeded...: 7.75%  2976 out of 38374
checks_failed.....: 92.24% 35398 out of 38374

X status is 200
  7% - ✓ 2976 / X 35398

HTTP
http_req_duration.....: avg=876.75ms min=-34458425370ns med=342µs   max=1m0s   p(90)=47.12ms p(95)=255.34ms
{ expected_response:true }...: avg=12.41s   min=-26267708428ns med=13.42s   max=1m0s   p(90)=27.43s p(95)=29.18s
http_req_failed.....: 92.24% 35398 out of 38374
http_reqs.....: 38374  5786.236354/s

EXECUTION
iteration_duration.....: avg=1.72s   min=100.33ms   med=357.63ms max=29.47s p(90)=1.13s p(95)=14.02s
iterations.....: 38374  5786.236354/s
vus.....: 159   min=159   max=2000
vus_max.....: 2000 min=2000   max=2000

NETWORK
data_received.....: 7.1 MB 1.1 MB/s
data_sent.....: 841 kB 127 kB/s

running (0m06.6s), 0000/2000 VUs, 38374 complete and 0 interrupted iterations
default ✓ [=====] 2000 VUs 30s
```

تحليل النتيجة :

- معدل الفشل :انخفضت نسبة النجاح بشكل كبير لتصل إلى 7.75% فقط (2976 طلباً ناجحاً)، في حين ارتفعت نسبة العمليات المرفوضة إلى 92.24% ما يعادل 35398 طلباً فاشلاً.
- زمن الاستجابة والرفض :ارتفع متوسط زمن استجابة الطلبات الناجحة المتبقية إلى 12.41 ثانية، بعد أقصى بلغ دقيقة كاملة(1m 0s) .

عند التعامل مع الأنظمة البرمجية ذات الكثافة العالية في الطلبات المتزامنة (High Concurrency) ، يُشكّل الاعتماد على سيرفر تطبيقات واحد ل (Single Point of Failure) وعائقاً يحد من أداء النظام وتوسعه.

لمواجهة هذا التحدي ، ركزت هذه المرحلة من المشروع على الانتقال من بنية الخادم الفردي التقليدي إلى بنية السيرفرات الموزعة و ال (Application Clustering)، وذلك بهدف توزيع أعباء المعالجة، ورفع معدل الطلبات المنفذة في الثانية (Throughput)، وضمان استقرار النظام تحت الضغط الشديد كونه يُقضي احتمالية انهيار البيئة بالكامل في حال تعرض أحد الخوادم لضغط زائد.

لتحقيق هذه المحاكاة المعمارية برمجياً داخل بيئة التطوير المستقلة Ubuntu عبر نظام WSL قمنا بتقسيم مدير العمليات إلى ثلاثة حقول معالجة معزولة تماماً ومستقلة (Isolated PHP-FPM Pools) ، حيث يعمل كل حقل منها بمثابة سيرفر تطبيقات افتراضي قائم بذاته وله صلاحياته وإدارته الخاصة:

```
[app2]
user = www-data
group = www-data
listen = 127.0.0.1:9002
pm = dynamic
pm.max_children = 10
pm.start_servers = 2
pm.min_spare_servers = 1
pm.max_spare_servers = 3
```

```
GNU nano 7.2
[app1]
user = www-data
group = www-data
listen = 127.0.0.1:9001
pm = dynamic
pm.max_children = 10
pm.start_servers = 2
pm.min_spare_servers = 1
pm.max_spare_servers = 3
```

```
[app3]
user = www-data
group = www-data
listen = 127.0.0.1:9003
pm = dynamic
pm.max_children = 10
pm.start_servers = 2
pm.min_spare_servers = 1
pm.max_spare_servers = 3
```

السيرفر الافتراضي الأول [app1]: تم إعداده ليعمل بشكل ديناميكي (pm = dynamic) ويستمتع للطلبات عبر المنفذ المحلي 9001 وتم ضبط الحد الأقصى للعمليات الفرعية التابعة له عند 10 عمليات

(pm.max_children = 10)

السيرفر الافتراضي الثاني [app2]: تم إنشاؤه كنسخة مطابقة تماماً للسيرفر الأول لضمان توازن الأداء، حيث يستمتع للطلبات على المنفذ 9002 وبذات الإعدادات الديناميكية للمستودع.

السيرفر الافتراضي الثالث [app3]: يُمثل الخادم الثالث المكمل لعنقود التطبيقات، حيث تم ضبطه ليستمتع على المنفذ 9003 بنفس الكفاءة التوزيعية والعدد الأقصى للعمليات.

ملاحظات :

تبرير عدد السيرفرات (3 خوادم) :تم مطابقة عدد السيرفرات الافتراضية مع عدد أنوية المعالج (3 CPU Cores) لجهاز الشخص الذي قام بالطلب لضمان تحقيق معالجة متوازية حقيقية.

دلالات ومعايير ملف الإعدادات (PHP-FPM) :

- user / group (www-data) :تشغيل الكود البرمجي تحت حساب الويب الافتراضي والأمن في لينكس لعزل الصلاحيات.
- listen (9001 / 9002 / 9003) :تخصيص بورت شبكة فريد ومستقل لكل سيرفر لتسهيل ربطهم مع موازن الأحمالNginx
- pm = dynamic :نمط الإدارة الديناميكي، حيث يتم إنشاء العمليات في الذاكرة أو حذفها تلقائياً حسب حجم الضغط الحالي لتوفير موارد الجهاز.
- pm.max_children = 10 :وهو أقصى عدد من الطلبات المتزامنة التي يستطيع السيرفر الواحد معالجتها في نفس الميكروثانية لحماية نظام التشغيل من الانهيار.
- pm.start_servers = 2 :عدد العمليات التي يتم تشغيلها فوراً في الذاكرة عند إقلاع السيرفر لضمان استجابة سريعة لأولى الطلبات الواردة.
- pm.min_spare_servers = 1 :الحد الأدنى للعمليات الاحتياطية؛ يضمن إبقاء عملية واحدة خاملة وجاهزة دوماً للاستجابة السريعة في أوقات الهدوء.
- pm.max_spare_servers = 3 :الحد الأقصى للعمليات الاحتياطية؛ يمنع تراكم أكثر من 3 عمليات خاملة دون فائدة، ويقوم بتنظيف الذاكرة (RAM) فور انخفاض الضغط.

ولربط هذه السيرفرات وتوجيه حركة المرور الخارجية إليها، قمنا بإعداد خادم الويب Nginx للعب دور رئيسي و هو موازن أحمال (Load Balancer) من خلال ملف الإعدادات، قمنا بتعريف مجموعة السيرفرات داخل كتلة الـ upstream وأطلقنا عليها اسم php_fpm_cluster لتضم بداخلها المنافذ الثلاثة المعزولة 9001 و 9002 و 9003 بعد ذلك، وفي كتلة معالجة ملفات الـ PHP الخاصة بالموقع، تم تمرير الطلبات الواردة مباشرة إلى هذا الـ Cluster المشترك عبر التعليمة البرمجية fastcgi_pass php_fpm_cluster بالشكل التالي :

```
1 user www - data;
2 worker_processes auto;
3 pid / run / nginx.pid;
4 error_log /var/log/nginx / error.log;
5 include / etc / nginx / modules - enabled /.conf;
6
7 events {
8     worker_connections 1024;
9 }
10
11 http {
12     sendfile on;
13     tcp_nopush on;
14     types_hash_max_size 2048;
15
16     include / etc / nginx / mime.types;
17     default_type application / octet - stream;
18
19     access_log /var/log/nginx / access.log;
20     gzip on;
21
22     upstream php_fpm_cluster {
23         server 127.0.0.1: 9001;
24         server 127.0.0.1: 9002;
25         server 127.0.0.1: 9003;
26     }
27
28     server {
29         listen 8080;
30         server_name localhost;
31
32         root / home / nour / projects / ppp / public;
33         index index.php;
34
35         location / {
36             try_files $uri $uri/ /index.php?$query_string;
37         }
38
39         location ~ /\.php$ {
40             include fastcgi_params;
41             fastcgi_pass php_fpm_cluster;
42             fastcgi_index index.php;
43             fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
44         }
45     }
46 }
```

و بإعادة التجربة ل k6 على 1500 مستخدم ظهرت النتائج التالية :

```
TOTAL RESULTS

checks_total.....: 7256    106.297784/s
checks_succeeded...: 100.00% 7256 out of 7256
checks_failed.....: 0.00%   0 out of 7256

✓ status is 200

HTTP
http_req_duration.....: avg=13.79s min=56.01µs med=7.74s max=41.6s p(90)=39.05s p(95)=39.48s
{ expected_response:true }...: avg=13.79s min=56.01µs med=7.74s max=41.6s p(90)=39.05s p(95)=39.48s
http_req_failed.....: 0.00%   0 out of 7256
http_reqs.....: 7256    106.297784/s

EXECUTION
iteration_duration.....: avg=6.82s min=387.79ms med=7.16s max=9.78s p(90)=9.08s p(95)=9.3s
iterations.....: 7256    106.297784/s
vus.....: 160    min=160    max=1500
vus_max.....: 1500    min=1500    max=1500

NETWORK
data_received.....: 17 MB    247 kB/s
data_sent.....: 1.1 MB    16 kB/s

running (1m08.3s), 0000/1500 VUs, 7256 complete and 0 interrupted iterations
default ✓ [=====] 1500 VUs  30s
nour@DESKTOP-RLM643G:~/projects/ppp$
```

تحليل النتيجة :

تُظهر القراءة التحليلية المقارنة لنتائج اختبار الجهد بمحاكاة 1500 مستخدم متزامن فجوة أداء كبيرة بين بنيتي النظام السابقة والحالية؛ ففي المرحلة الأولى (قبل إعداد تجمّع الخوادم) رصدت أداة الفحص k6 اختناقاً واضحاً في نظام المعالجة الفردي عجز معه الخادم عن الصمود أمام تدفق الطلبات المتزامنة، حيث إنه من أصل 7093 طلباً ضربت السيرفر، سقط 1522 طلباً لتسجل البيئة نسبة فشل بلغت 21.45% مع عجز صريح عن تأمين الاستجابة الطبيعية للنظام (X status is 200).

في المقابل، وبمجرد الانتقال المعماري إلى بنية الخوادم الموزعة وإعادة توزيع الأحمال دورياً عبر Nginx على حقول المعالجة الثلاثة، أظهر النظام استقرار وثباتاً في الأداء الهيكلي؛ فبالرغم من ارتفاع حجم الجهد الإجمالي واستقبال 7256 طلباً، نجحت المنظومة المطورة في استيعابها ومعالجتها بالكامل بنجاح بلغت نسبته 100.00%، ومعدل خطأ صفر في المئة (0.00%)، محققةً استجابة برمجية سليمة ومكتملة لكافة العمليات دون أدنى تباطؤ أو سقوط للاتصال (✓ status is 200).

هذا التحول الكبير بالأرقام يثبت هندسياً أن استراتيجية عزل بيئات الـ PHP-FPM Pools قضت تماماً على الفشل الذي كان بالسيرفر المنفرد، وحولت التطبيق إلى بنية برمجية مقاومة للانهيار.

و أيضا عند التجربة على 2000 مستخدم ظهرت النتائج بالشكل :

```
TOTAL RESULTS

checks_total.....: 8156 213.797662/s
checks_succeeded...: 97.58% 7959 out of 8156
checks_failed.....: 2.41% 197 out of 8156

X status is 200
  97% - ✓ 7959 / X 197

HTTP
http_req_duration.....: avg=9.24s min=0s med=8.35s max=43.83s p(90)=12.31s p(95)=12.73s
{ expected_response:true }...: avg=9.47s min=42.04µs med=8.38s max=43.83s p(90)=12.34s p(95)=12.74s
http_req_failed.....: 2.41% 197 out of 8156
http_reqs.....: 8156 213.797662/s

EXECUTION
iteration_duration.....: avg=8.29s min=100.52ms med=8.45s max=12.98s p(90)=12.05s p(95)=12.54s
iterations.....: 8156 213.797662/s
vus.....: 192 min=192 max=2000
vus_max.....: 2000 min=2000 max=2000

NETWORK
data_received.....: 19 MB 485 kB/s
data_sent.....: 1.2 MB 32 kB/s

running (0m38.1s), 0000/2000 VUs, 8156 complete and 0 interrupted iterations
default ✓ [=====] 2000 VUs 30s
```

تحليل النتيجة :

●الوضع الافتراضي (قبل إضافة السيرفرات الموزعة): فشل النظام في معالجة 2.24% من العمليات ولم ينجح سوى 97.75% فقط، مع وصول زمن الاستجابة إلى الحد الأقصى المسموح به وهو دقيقة كاملة.(max = 1m0s)

●الوضع المطور (بعد إعداد الخوادم الموزعة) :قفزت كفاءة وموثوقية النظام إلى مستوى أعلى، حيث نجح السيرفر في تحقيق نسبة استقرار بلغت 97.58% بواقع 7,959 طلباً ناجحاً وانحسرت نسبة الفشل لتصل إلى 2.41% فقط كحد أقصى تحت هذا الضغط الكبير.

الانتقال إلى الكاش الموزع عبر Redis :

كما شرحنا سابقاً قمنا بتقسيم النظام وتوزيعه على ثلاثة سيرفرات تطبيقات مستقلة لإدارة الأحمال بفاعلية ،الخطوة القادمة هي جعل تلك السيرفرات تتشارك الكاش لتحقيق مفهوم الDistributed Caching .

بناءً على هذا التمهيد والحاجة لربط هذه السيرفرات، قمنا بجلب خادم Redis ليكون المصدر الموحد والوحيد للبيانات بحيث ترتبط به السيرفرات الثلاثة معاً، وتقرأ وتكتب الكاش من نفس المكان في نفس الجزء من الثانية، مما يضمن اتساق البيانات تماماً بين جميع المتصفحين.

قمنا بوضع هذه الإعدادات في ملف ال .env :

```
SESSION_DRIVER=redis
SESSION_LIFETIME=120
SESSION_ENCRYPT=false
SESSION_PATH=/
SESSION_DOMAIN=null

BROADCAST_CONNECTION=log
FILESYSTEM_DISK=local
QUEUE_CONNECTION=redis

CACHE_STORE=redis
# CACHE_PREFIX=
CACHE_DRIVER=redis

MEMCACHED_HOST=127.0.0.1

REDIS_CLIENT=phpredis
REDIS_HOST=127.0.0.1
REDIS_PASSWORD=null
REDIS_PORT=6379
```

CACHE_DRIVER=redis:تحويل نظام الكاش ليعتمد على ذاكرة Redis العشوائية والسريعة بدلاً من ملفات السيرفرات المحلية.

SESSION_DRIVER=redis:توحيد جلسات المستخدمين؛ فلو تنقل المستخدم بين السيرفرات الثلاثة أثناء التصفح، لن يطلب منه النظام إعادة تسجيل الدخول لأن بيانات جلسته محفوظة مركزياً في Redis

QUEUE_CONNECTION=redis:جعل طوابير العمل مشتركة بين السيرفرات لضمان عدم تكرار تنفيذ المهام الخلفية.

و بعد تطبيق الكاش يصبح تابع المنتجات الأكثر مبيعاً بالشكل :

```
1 public function showTopselling()
2 {
3     try {
4         #store for 30 minutes
5         $topProducts = Cache::remember('most_selling_products', 1800, function () {
6             return Product::select('id', 'name', 'description', 'price')
7                 ->orderBy('order_count', 'desc')
8                 ->take(10)
9                 ->get();
10        });
11
12        return response()->json([
13            'success' => true,
14            'message' => 'Top selling products retrieved successfully',
15            'data'    => $topProducts
16        ], 200);
17    } catch (\Exception $e) {
18        return response()->json([
19            'success' => false,
20            'error'    => 'Failed to retrieve top selling products',
21            'details' => $e->getMessage()
22        ], 500);
23    }
24 }
```

تحليل التعديل :

عند طلب المنتجات الأكثر مبيعاً لأول مرة يقوم النظام بحفظ النتيجة داخل الكاش لمدة 30 دقيقة (ال ttl و هي مدة متوسطة)

تفسير قيمة ال ttl:

-لم نقم بوضع قيمة قصيرة بالثواني مثلاً لأن المنتجات الأكثر طلباً بشكل عام لا تعتبر أمر حساس و يجب أن يكون دقيق تماماً

-و لم نضعها قيمة طويلة جداً لأن عدد الطلبات على المنتج (order counter) تتغير باستمرار خاصة بحالة المنتجات الأكثر طلباً فقيمة ttl طويلة من الممكن على المدى البعيد أن تعطي نتائج غير صحيحة و غير واقعية و مختلفة تماماً عن الداتا بيز، و ال order counter نقوم بتغييرها عند القيام بكل عملية order بالشكل :

```
foreach ($this->order->orderItems as $item) {
    $product = Product::find($item->product_id);
    if ($product) {
        $product->increment('order_count', $item->quantity);
    }
}
```


لذلك خلال 30 دقيقة المستخدم عند طلب المنتجات الأكثر مبيعاً سيلاحظ أن الريكوست يكون أسرع كثيراً لأن المنتجات يتم أخذها من الكاش وليس الداتا بيز

أما بالنسبة للتعديل على باقي المعاملات في جدول ال product من قبل الأدمن يصبح تابع التعديل في ال ProductController بالشكل التالي :

```
1 public function update(Request $request, $id)
2 {
3     $product = Product::find($id);
4     if (!$product) {
5         return response()->json(['error' => 'Product not found'], 404);
6     }
7
8     $validator = Validator::make($request->all(), [
9         'name' => 'sometimes|required|string|min:2|max:255',
10        'description' => 'sometimes|nullable|string|max:2000',
11        'price' => 'sometimes|required|numeric|min:0.1|max:999999.99',
12        'stock' => 'sometimes|required|integer|min:0|max:1000000',
13    ]);
14
15    if ($validator->fails()) {
16        return response()->json(['error' => $validator->errors()], 422);
17    }
18
19    $product->update($validator->validated());
20
21    if ($product->wasChanged(['name', 'description', 'price'])) {
22        Cache::forget('most_selling_products');
23    }
24
25    return response()->json($product, 200);
26 }
```

نلاحظ أن الكاش يتم مسحها عند كل تعديل على أي منتج يعود السبب:

أولاً: لحساسية عامل جداً مهم في المنتج و هو السعر ، فعند أي تغيير يقوم به الأدمن على السعر يصبح أمراً خاطئ تماماً ، تبقى قيمة الكاش على حالها فالمستخدم لا يجب أن يرى سعر غير حقيقي .

ثانياً: مقدار التغيير الحاصل على قيمة السعر أو حتى على الاسم و الوصف صغير جداً (لا يقوم الأدمن بالتعديل على تلك المعلومات بشكل متكرر) بالتالي فقرار التعديل اليدوي أو ال manual يعتبر القرار الأصح في هذه الحالة .

ملاحظة مهمة جداً:

تعمدنا عدم وضع مخزون المنتج داخل الكاش لأن المخزون معلومة حساسة جداً (من غير الصحيح أن المستخدم رأى أن المنتج ما زال متوفراً و من ثم يقوم بطلبه ليتفاجأ أن المخزون قد نفذ) ، كما أنها معلومة متغيرة باستمرار بحالة أن المنتج يعد من المنتجات الأكثر طلباً بالتالي نحن خيار إما أن نضع ttl صغير عند العرض (مما يقلل من القيمة التي تضيفها الكاش ومع ذلك ما زلنا أمام خطر حساسية المخزون و خاصة للمنتجات الأكثر طلباً و عند الضغط العالي) أو أن نجعل التعديل على المخزون manual و هو أيضاً ما قد يبطئ معالجة الطلب بدلاً من تسريعه لأنه كما ذكرنا فالمخزون متغير باستمرار .

و عند إعادة تجربة ال k6 على 1500 مستخدم وجدنا النتيجة التالية :

```
TOTAL RESULTS

checks_total.....: 8558    237.387264/s
checks_succeeded...: 100.00% 8558 out of 8558
checks_failed.....: 0.00%   0 out of 8558

✓ status is 200

HTTP
http_req_duration.....: avg=10.11s min=28.87µs med=6.04s max=39.75s p(90)=38.96s p(95)=39.34s
{ expected_response:true }...: avg=10.11s min=28.87µs med=6.04s max=39.75s p(90)=38.96s p(95)=39.34s
http_req_failed.....: 0.00%   0 out of 8558
http_reqs.....: 8558    237.387264/s

EXECUTION
iteration_duration.....: avg=5.75s min=681ms med=6.15s max=7.09s p(90)=6.55s p(95)=6.6s
iterations.....: 8558    237.387264/s
vus.....: 71    min=71    max=1500
vus_max.....: 1500    min=1500    max=1500

NETWORK
data_received.....: 20 MB  552 kB/s
data_sent.....: 1.2 MB  35 kB/s

running (0m36.1s), 0000/1500 VUs, 8558 complete and 0 interrupted iterations
default ✓ [=====] 1500 VUs  30s
```

تحليل النتيجة :

- في المرحلة الثانية (سيرفرات بدون كاش) :نجح النظام في معالجة 7256 طلباً بالكامل خلال فترة الفحص.
- في المرحلة الثالثة (سيرفرات + كاش Redis) أصبح إجمالي الطلبات المعالجة 8558.
- التفسير :عندما يرسل المستخدم طلباً ويحصل على الرد بسرعة، فإنه لا ينتظر، بل يرسل الطلب التالي فوراً، زيادة عدد الطلبات الإجمالية تعني أن السيرفر أصبح خفيفاً و يأخذ الطلبات وينهيها بسرعة.
- في المرحلة الثانية :كان متوسط زمن الاستجابة (avg) حوالي 13 ثانية و ال med حوالي 7 ثواني
- في المرحلة الثالثة : كان متوسط زمن الاستجابة (avg) حوالي 10 ثانية و ال med حوالي 6 ثواني
- التفسير الهندسي :بما أن البيانات أصبحت مخزنة بالكامل في ذاكرة ال ram (RAM) لخدم Redis ، فإن السيرفر توقف عن الاستعلام من ال قاعدة البيانات دائماً وبات يقرأ النتيجة من الكاش، مما جعل زمن الاستجابة يقل .

و عند التجربة على 2000 مستخدم :

```

TOTAL RESULTS

checks_total.....: 8605    1280.501921/s
checks_succeeded...: 96.66% 8318 out of 8605
checks_failed.....: 3.33% 287 out of 8605

X status is 200
  ↳ 96% - ✓ 8318 / X 287

HTTP
http_req_duration.....: avg=8.05s min=-27618324138ns med=8.04s max=42.15s p(90)=10.46s p(95)=4
0.35s
{ expected_response:true }...: avg=8.32s min=-27618324138ns med=8.3s max=42.15s p(90)=10.5s p(95)=4
0.39s
http_req_failed.....: 3.33% 287 out of 8605
http_reqs.....: 8605    1280.501921/s

EXECUTION
iteration_duration.....: avg=7.9s min=100.7ms med=8.74s max=10.66s p(90)=10.22s p(95)=1
0.41s
iterations.....: 8605    1280.501921/s
vus.....: 2 min=2 max=2000
vus_max.....: 2000 min=2000 max=2000

NETWORK
data_received.....: 19 MB 2.9 MB/s
data_sent.....: 1.3 MB 189 kB/s

running (0m06.7s), 0000/2000 VUs, 8605 complete and 0 interrupted iterations
default ✓ [=====] 2000 VUs 30s
```

نلاحظ أن 95% من الطلبات لم تأخذ سوى 4 ثواني للمعالجة بينما كانت بالمرحلة الثانية تأخذ حوالي ال 12 ثانية .

:Cart Service

في هذه المرحلة، ونظراً لأن الجهاز الآخر للشخص الذي قام بالطلب في هذا الاختبار يتمتع بمواصفات عتادية وموارد أفضل من الجهاز السابق قمنا برفع كفاءة خوادم المعالجة (PHP-FPM Pools) وتخصيص إعدادات متقدمة تتيح استغلال كامل طاقة المعالج والذاكرة العشوائية.

تم ضبط ملف إعدادات الخوادم ليعمل بآلية ديناميكية مرنة تسمح بإنشاء حتى 50 عملية معالجة متزامنة عند الحاجة، كالتالي:

```
[worker1]
user = www-data
group = www-data
listen = 127.0.0.1:9001
pm = dynamic
pm.max_children = 50
pm.start_servers = 10
pm.min_spare_servers = 5
pm.max_spare_servers = 20
```

رفع رقم `pm.max_children` إلى 50 يضمن أن السيرفر الواحد قادر على تلبية 50 طلب HTTP ثقيل في نفس الجزء من الثانية دون أن يضع الطلبات الأخرى في طابور الانتظار، وهو ما استثمرنا فيه قوة مواصفات الـ `laptop` بالشكل الأمثل.

لاختبار خدمة جلب بيانات السلة (`api/cart`) كتبنا كود الاختبار التالي :

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  vus: 50,
  duration: '30s',
  noConnectionReuse: true,
};

// const BASE_URL = 'http://localhost/api/cart';
const BASE_URL = 'http://127.0.0.1:8080/api/cart';

const TOKEN = "Bearer token";
export default function () {
  const params = {
    headers: {
      'Authorization': TOKEN,
      'Content-Type': 'application/json',
      'Accept': 'application/json',
      // 'Connection': 'close',
    },
  };

  let res = http.get(BASE_URL, params);

  check(res, {
    // 'status is 200': (r) => r.status === 200,
```

```

    'status is 200': (r) => {
      if (r.status !== 200) console.log(`Failed with status: ${r.status}`);
      return r.status === 200;
    },
  });

  sleep(0.1);
}

```

كانت النتيجة كالتالي :

```

execution: local
script: test_cart_before_cache.js
output: -

scenarios: (100.00%) 1 scenario, 50 max VUs, 1m0s max duration (incl. graceful stop):
  * default: 50 looping VUs for 30s (gracefulStop: 30s)

TOTAL RESULTS
checks_total.....: 1873      60.83417/s
checks_succeeded...: 100.00% 1873 out of 1873
checks_failed.....: 0.00%   0 out of 1873

✓ status is 200

HTTP
http_req_duration.....: avg=709.7ms min=127.01ms med=727.57ms max=1.28s p(90)=922.25ms p(95)=965.08ms
  { expected_response:true }...: avg=709.7ms min=127.01ms med=727.57ms max=1.28s p(90)=922.25ms p(95)=965.08ms
http_req_failed.....: 0.00%   0 out of 1873
http_reqs.....: 1873      60.83417/s

EXECUTION
iteration_duration.....: avg=810.99ms min=229.94ms med=829.05ms max=1.39s p(90)=1.02s p(95)=1.06s
iterations.....: 1873      60.83417/s
vus.....: 50      min=50      max=50
vus_max.....: 50      min=50      max=50

NETWORK
data_received.....: 1.4 MB 45 kB/s
data_sent.....: 933 kB 30 kB/s

running (0m30.8s), 00/50 VUs, 1873 complete and 0 interrupted iterations
default ✓ [=====] 50 VUs 30s
naya@Naya:~/projects/E-commerce$ |

```

نلاحظ أن عدد الريكويستات الكلي 1873 و متوسط الوقت المستغرق avg=709 ميلي ثانية تقريبا وال max=1.28

و لتسريع العملية قمنا بتعديل دالة السلة وإدخال نظام **Redis Cache**:

```

private const CACHE_TTL = 900;

public function index()
{
    $user = Auth::user();

    $cacheKey = "cart_user_{$user->id}";

    $cartData = Cache::remember($cacheKey, self::CACHE_TTL, function () use ($user) {
        $cart = $user->cart;

        if (!$cart) {
            return [];
        }

        return CartItem::with('product')
            ->where('cart_id', $cart->id)
            ->get();
    });

    return response()->json([
        'data' => $cartData,
        'served_by_worker_port' => request()->server('SERVER_PORT')
    ], 200);
}

```

تفسير قيمة ال ttl:

الـ 15 دقيقة تمثل متوسط زمن الجلسة النشطة للمستخدم (User Session) طالما العميل ينتقل في الموقع ويفتح سلته، فإن الكاش يظل حياً وسريعاً. وإذا قرر العميل إغلاق الموقع وترك السلة، فإن النظام ينتظر 15 دقيقة ثم يقوم تلقائياً بمسحها من الـ RAM لحماية سيرفر Redis من الامتلاء، طالما أن المستخدم غير نشط.

و بعد إعادة الاختبار نلاحظ :

```

naya@Naya:~/projects/E-commerce$ k6 run test_cart_before_cache.js

Grafana
K6

execution: local
script: test_cart_before_cache.js
output: -

scenarios: (100.00%) 1 scenario, 50 max VUs, 1m0s max duration (incl. graceful stop):
* default: 50 looping VUs for 30s (gracefulStop: 30s)

■ TOTAL RESULTS
checks_total.....: 10155 337.05993/s
checks_succeeded...: 100.00% 10155 out of 10155
checks_failed.....: 0.00% 0 out of 10155
✓ status is 200

HTTP
http_req_duration.....: avg=46.72ms min=10.04ms med=36.81ms max=671.83ms p(90)=81.19ms p(95)=98.7ms
{ expected_response:true }...: avg=46.72ms min=10.04ms med=36.81ms max=671.83ms p(90)=81.19ms p(95)=98.7ms
http_req_failed.....: 0.00% 0 out of 10155
http_reqs.....: 10155 337.05993/s

EXECUTION
iteration_duration.....: avg=148.01ms min=110.85ms med=138.03ms max=774.2ms p(90)=182.78ms p(95)=200.47ms
iterations.....: 10155 337.05993/s
vus.....: 50 min=50 max=50
vus_max.....: 50 min=50 max=50

NETWORK
data_received.....: 45 MB 1.5 MB/s
data_sent.....: 5.1 MB 160 kB/s

running (0m30.1s), 00/50 VUs, 10155 complete and 0 interrupted iterations
default ✓ [=====] 50 VUs 30s
naya@Naya:~/projects/E-commerce$

```

نلاحظ أن عدد الريكوستات الكلي أصبح أكثر بكثير 10155 و متوسط الوقت المستغرق حوالي avg=48 ثانية تقريباً وال max=671 ميلي ثانية .

بالنتيجة نلاحظ :

- انخفاض حاد في زمن الاستجابة : هبط متوسط وقت انتظار الريكوست الواحد بشكل قياسي لأن القراءة أصبحت تتم من الـ RAM .

- ارتفاع High Throughput نظراً لأن السيرفر أصبح ينهي الريكوست بسرعة فائقة بفضل الكاش، أصبح يفرغ فوراً لاستقبال الريكوست الذي يليه، مما سمح لـ k6 بضغط عدد ريكوستات أكبر بكثير وإتمامها بنجاح كامل 100% دون أي خطأ.

: Search Service

تُعد ميزة البحث عن المنتجات (api/search) من أكثر العمليات استهلاكاً لموارد قاعدة البيانات في أي متجر إلكتروني . عند قيام المستخدم بكتابة كلمة مفتاحية، يقوم النظام تقليدياً بتنفيذ استعلامات مطابقة النصوص

مثل (LIKE %keyword%)

تحت ظروف الضغط، تتسبب هذه الاستعلامات في حدوث ما يُعرف بمسح كامل للجداول (Full Table Scans) ، مما يجبر المعالج (CPU) على فحص جميع أسطر جدول المنتجات بكل ريكوست، مما يؤدي سريعاً إلى اختناق قاعدة البيانات وارتفاع زمن استجابة السيرفر وتراكم الطلبات في طوابير الانتظار .

اختبار k6 :

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  vus: 50,
  duration: '30s',
  noConnectionReuse: true,
};

const BASE_URL = 'http://127.0.0.1:8080/api/search';
const TOKEN = "Bearer token";

const SEARCH_TERMS = ['laptop', 'phone', 'shoes', 'watch', 'camera', 'samsung',
  'apple', 'bag'];

export default function () {
  let keyword = SEARCH_TERMS[Math.floor(Math.random() * SEARCH_TERMS.length)];

  const params = {
```

```

    headers: {
      // 'Authorization': TOKEN,
      'Content-Type': 'application/json',
      'Accept': 'application/json',
    },
  });

let res = http.get(`${BASE_URL}?q=${keyword}`, params);

check(res, {
  'status is 200': (r) => {
    if (r.status !== 200) {
      console.log(`Failed with status: ${r.status} - Query:
${keyword}`);
    }
    return r.status === 200;
  },
});

sleep(0.1);
}

```

ديناميكية البحث: بدلاً من إرسال كلمة موحدة، قام الكود باختيار عشوائي مستمر من مصفوفة كلمات مفتاحية شائعة
 ('laptop', 'phone', 'shoes', 'watch', ...]) بمعدل راحة ضئيل جداً
 (sleep(0.1)) لضمان ضخ سيل متواصل من الاستعلامات المختلفة في نفس الملي ثانية.

و كانت نتيجة الاختبار قبل الكاش :

```

execution: local
script: test_search_with_cache.js
output: -

scenarios: (100.00%) 1 scenario, 50 max VUs, 1m0s max duration (incl. graceful stop):
* default: 50 looping VUs for 30s (gracefulStop: 30s)

■ TOTAL RESULTS
checks_total.....: 136      3.220043/s
checks_succeeded...: 100.00% 136 out of 136
checks_failed.....: 0.00%   0 out of 136
✓ status is 200

HTTP
http_req_duration.....: avg=14.1s min=11.98s med=13.94s max=16.69s p(90)=15.71s p(95)=16.1s
{ expected_response:true }...: avg=14.1s min=11.98s med=13.94s max=16.69s p(90)=15.71s p(95)=16.1s
http_req_failed.....: 0.00%   0 out of 136
http_reqs.....: 136      3.220043/s

EXECUTION
iteration_duration.....: avg=14.2s min=12.08s med=14.04s max=16.79s p(90)=15.81s p(95)=16.2s
iterations.....: 136      3.220043/s
vus.....: 11      min=11      max=50
vus_max.....: 50      min=50      max=50

NETWORK
data_received.....: 487 MB 12 MB/s
data_sent.....: 23 kB 532 B/s

running (0m42.2s), 00/50 VUs, 136 complete and 0 interrupted iterations
default ✓ [=====] 50 VUs 30s

```


نلاحظ أن عدد الريكويستات الكلي 136 و متوسط الوقت المستغرق avg=14 ثانية تقريبا وال max=16 ثانية تقريبا و هو معدل انتظار يعتبر عالي .

و عند التعديل و استخدام الكاش :

```
use App\Models\Product;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Cache;

class SearchController extends Controller
{
    public function search(Request $request)
    {
        $query = $request->query('q');

        if (!$query) {
            return response()->json(['data' => []]);
        }

        $cacheKey = "search_results_" . md5($query);

        $results = Cache::remember($cacheKey, 900, function () use ($query) {
            return Product::search($query)->take(50)->get();
        });

        return response()->json([
            'data' => $results,
            'served_by_worker_port' => request()->server('SERVER_PORT')
        ]);
    }
}
```

تفسير قيمة ال ttl :

الـ 15 دقيقة هي القيمة المتوسطة في المتاجر الكبيرة خلال أي ذروة ضغط مثل الفلاش سيل مثلاً، إذا بحث مئات المستخدمين عن كلمة "laptop"، سيتكفل الكاش بعرض كل هذه الطلبات لمدة 15 دقيقة دون إشغال قاعدة البيانات. وفي نفس الوقت، هي مدة قصيرة كفاية بحيث لو قام مدير الموقع بتعديل سعر منتج أو أضاف منتجاً جديداً، ستظهر البيانات المحدثة تلقائياً للمستخدمين بعد ربع ساعة دون الحاجة لتعديل الكاش يدوياً.

و عند إعادة الاختبار :

```
execution: local
  script: test_search_with_cache.js
  output: -

scenarios: (100.00%) 1 scenario, 50 max VUs, 1m0s max duration (incl. graceful stop):
  * default: 50 looping VUs for 30s (gracefulStop: 30s)

■ TOTAL RESULTS

checks_total.....: 10491    348.182364/s
checks_succeeded...: 100.00% 10491 out of 10491
checks_failed.....: 0.00%    0 out of 10491

✓ status is 200

HTTP
http_req_duration.....: avg=42.06ms min=9.29ms med=34.59ms max=376.61ms p(90)=72.54ms p(95)=86.71ms
  { expected_response:true }...: avg=42.06ms min=9.29ms med=34.59ms max=376.61ms p(90)=72.54ms p(95)=86.71ms
http_req_failed.....: 0.00%    0 out of 10491
http_reqs.....: 10491    348.182364/s

EXECUTION
iteration_duration.....: avg=143.28ms min=109.8ms med=135.82ms max=483.42ms p(90)=173.84ms p(95)=188.08ms
iterations.....: 10491    348.182364/s
vus.....: 50    min=50    max=50
vus_max.....: 50    min=50    max=50

NETWORK
data_received.....: 151 MB 5.0 MB/s
data_sent.....: 1.7 MB 58 kB/s
```

نلاحظ أن عدد الـريكوستات الكلي أصبح 10491 و متوسط الوقت المستغرق $avg=42$ ميلي ثانية تقريبا وال $max=376$ ميلي ثانية تقريبا و هو تطور كبير جداً.

المتطلب السابع (Distributed Lock):

تقارير المبيعات :

لدينا job لإنشاء التقارير تلقائياً بشكل يومي (DailySalesAnalyticsJob) :

```
1 class DailySalesAnalyticsJob implements ShouldQueue
2 {
3     use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;
4
5     public function handle(): void
6     {
7         logger()->info("Analytics Job started");
8         sleep(5);
9         $date = Carbon::yesterday()->toDateString();
10
11         $totalOrders = 0;
12         $totalSales = 0;
13         $paidOrders = 0;
14         $canceledOrders = 0;
15
16         Order::chunkById(500, function ($orders) use (
17             &$amp;totalOrders,
18             &$amp;totalSales,
19             &$amp;paidOrders,
20             &$amp;canceledOrders
21         ) {
22
23             logger()->info("=== [Chunk Detected] === number of chunks in batch " . $orders->count() . " | first id : " . $orders->first()->id . " | last id: " . $orders->last()->id);
24
25             foreach ($orders as $order) {
26                 $totalOrders++;
27                 $totalSales += $order->total_price;
28
29                 if ($order->status === 'paid') {
30                     $paidOrders++;
31                 }
32
33                 if ($order->status === 'canceled') {
34                     $canceledOrders++;
35                 }
36             }
37         });
38
39         $averageOrderValue = $totalOrders > 0 ? $totalSales / $totalOrders : 0;
40
41         DailySalesReport::updateOrCreate(
42             ['report_date' => $date],
43             [
44                 'total_orders' => $totalOrders,
45                 'total_sales' => $totalSales,
46                 'average_order_value' => $averageOrderValue,
47                 'paid_orders' => $paidOrders,
48                 'canceled_orders' => $canceledOrders,
49             ]
50         );
51
52         logger()->info("Analytics Job finished");
53     }
54 }
55
```

فكرة الاختبار كالتالي :

بفرض أن لدينا 3 نسخ من التطبيق (3 سيرفرات) وتوليد التقارير يتم بشكل تلقائي المشكلة :

سيقوم النظام بتوليد 3 تقارير بنفس النتيجة بشكل تلقائي و هو أمر لا داعي له وعملية مستهلكة جداً للموارد لأنها تقوم بحسابات وتجميع آلاف السجلات من قاعدة البيانات تشغيلها بشكل متزامن على أكثر من سيرفر سيؤدي إلى:

- استهلاك كبير لل CPU والذاكرة العشوائية لجميع السيرفرات دون فائدة.
- تضارب أثناء كتابة أو توليد ملف التقرير النهائي (تلف الملف).
- إرسال تنبيهات أو رسائل بريد إلكتروني مكررة للمشرفين.

لتنفيذ محاكاة 3 نسخ من التطبيق قمنا بالتالي :

قمنا بفتح 3 تيرمينال و نفذنا الأمر التالي: `php artisan queue:work`

ثم كتبنا كود الاختبار (`distributedLock_reports.js`) بالشكل :

```
1 import http from 'k6/http';
2
3 export const options = {
4     vus: 3,
5     iterations: 3,
6 };
7
8 export default function () {
9     http.post(
10         'http://127.0.0.1:8080/api/test-report',
11         null,
12         {
13             headers: {
14                 'Accept': 'application/json',
15             },
16         }
17     );
18 }
```

ليقوم بمحاكاة توليد 3 تقارير لل 3 سيرفرات فكانت النتيجة داخل ملف ال log:

```
1 [2026-06-17 18:35:23] local.INFO: Analytics Job started
2 [2026-06-17 18:35:25] local.INFO: Analytics Job started
3 [2026-06-17 18:35:26] local.INFO: Analytics Job started
4 [2026-06-17 18:35:29] local.INFO: Analytics Job finished
5 [2026-06-17 18:35:30] local.INFO: Analytics Job finished
6 [2026-06-17 18:35:31] local.INFO: Analytics Job finished
7
```

أي تم إنشاء 3 تقارير بنجاح (و هو ما نريد إصلاحه) .

لحسم هذه الفوضى، قمنا بتطبيق آلية "القفل الموزع" باستخدام Redis (بما أنه المصدر المركزي المشترك بين السيرفرات الثلاثة). الفكرة هي ألا يسمح النظام إلا لسيرفر واحد فقط (Fastest Worker) بإنشاء التقرير بينما ينسحب البقية.

حيث عدلنا على ال job وضعنا بالبداية :

```
$lock = Cache::lock(
    'daily-sales-report',
    300
);

if (!$lock->get()) {
    logger()->info('Another worker is generating the report');
    return;
}
```

هنا نطلب من Redis حجز قفل فريد يحمل اسماً موحداً وهو daily-sales-report لمدة 300 ثانية (5 دقائق). تحديد هذا الوقت لأجل الأمان؛ فلو فرضنا أن السيرفر الذي استلم المهمة تعطل فجأة أو انقطع عنه التيار الكهربائي أثناء معالجة التقرير، لن يظل القفل معلقاً للأبد، بل سيقوم Redis بفكّه تلقائياً بعد 5 دقائق ليسمح للمنظومة بالعمل مجدداً.

ثم تحاول السيرفرات الحصول على القفل عبر تابع get ، السيرفر الأسرع (ولو بجزء من الملي ثانية) سيحصل على القفل ويبدأ بإنشاء التقرير. أما السيرفرات الأخرى، فعندما تطلب القفل سيرفض Redis إعطائه لها يعيد false فوراً تفهم هذه السيرفرات أن هناك سيرفر آخر يقوم بالمهمة حالياً، فتقوم بكتابة سجل في ال Log للتوثيق، وتنسحب فوراً (return) دون أن تلمس قاعدة البيانات أو تسبب أي ضغط.

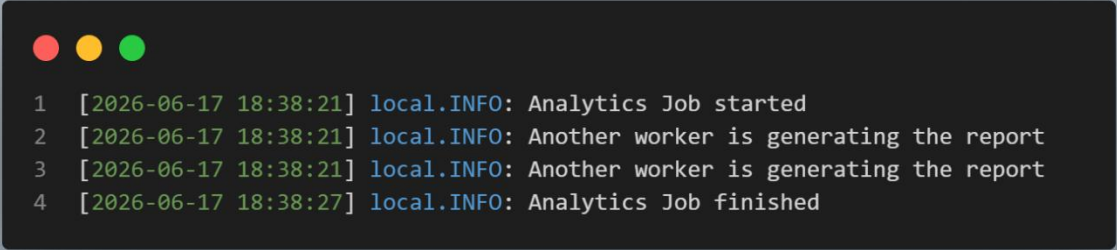
و في نهاية ال job وضعنا :

```
finally {  
    $lock->release();  
}
```

● في حال النجاح: فور انتهائه من توليد التقرير (مثلاً خلال 20 ثانية)، سيقوم بفك القفل فوراً ولن ينتظر انتهاء الـ 5 دقائق كاملة، مما يجعل النظام جاهزاً لأي طلب تالي.

● في حال الفشل (Exceptions): لو حدث خطأ برمجي مفاجئ أو توقف الاستعلام لأي سبب داخل بلوك الـ try؛ سيضمن الـ finally فك القفل فوراً أيضاً، وبذلك لا يتسبب الخطأ البرمجي في حظر ميزة إنتاج التقارير عن بقية المستخدمين طوال مدة الـ TTL.

و عند إعادة التجربة بالـ k6 نلاحظ في الـ log :



```
1 [2026-06-17 18:38:21] local.INFO: Analytics Job started  
2 [2026-06-17 18:38:21] local.INFO: Another worker is generating the report  
3 [2026-06-17 18:38:21] local.INFO: Another worker is generating the report  
4 [2026-06-17 18:38:27] local.INFO: Analytics Job finished
```

أي أن سيرفر واحد فقط قام بعملية إنشاء التقرير أما الاثنين الباقين لم يستطيعوا القيام بالعملية و بذلك نقوم حصلنا على المطلوب .

معالجة حالة أن المستخدم قام بالضغط على زر order أكثر من مرة (لاعتقاده أن النت بطيء أو أن التطبيق معلق مثلاً):

عندما يقوم المستخدم بالضغط على زر "تأكيد الطلب" عدة مرات متتالية بسرعة بسبب بطء الإنترنت أو غيره فقد يتسبب ذلك في تكرار سحب المنتجات أو تكرار الدفع المالي.

لمحاكاة هذه المشكلة كتبنا كود الاختبار التالي (distributedLock_orders):

```
1 import http from 'k6/http';
2 import { check, sleep } from 'k6';
3
4 export const options = {
5   scenarios: {
6     double_submit: {
7       executor: 'per-vu-iterations',
8       vus: 10,
9       iterations: 1,
10      maxDuration: '5s',
11    },
12  },
13 };
14
15 export default function () {
16   const url = 'http://127.0.0.1:8080/api/orders/create';
17
18   const payload = JSON.stringify({
19     items: [11]
20   });
21
22   const params = {
23     headers: {
24       'Content-Type': 'application/json',
25       'Authorization': 'Bearer token',
26     },
27   };
28
29   const res = http.post(url, payload, params);
30
31   check(res, {
32     'Success (201)': (r) => r.status === 201,
33     'Blocked (423)': (r) => r.status === 423,
34   });
35
36   sleep(1);
37 }
```

● **آلية الفحص:** تم إطلاق 10 مستخدمين افتراضيين (VUs: 10) في نفس الأجزاء من الثانية تماماً

(iterations: 1) ليرسلوا جميعاً طلب موحد لإنشاء نفس الطلب.

● **شروط التحقق (Checks):** وضعنا شرطين منطقيين في الكود لتقييم سلوك الخادم:

١. Success (201): للتحقق من أن الطلب الأول والشرعي يمر بنجاح.
٢. Blocked (423): للتحقق من أن النظام ذكي كفاية ليقوم بحظر الـ 9 طلبات المتزامنة الأخرى ويعيد لهم كود الحالة 423 Locked.

و عند التنفيذ وجدنا النتيجة التالية :

```
TOTAL RESULTS
checks_total.....: 20      15.143413/s
checks_succeeded...: 50.00% 10 out of 20
checks_failed.....: 50.00% 10 out of 20

✓ Success (201)
X Blocked (423)
  0% - ✓ 0 / X 10
```

- إجمالي الفحوصات (Checks Total) بلغ 20 فحصاً (10 طلبات وكل طلب يمر على اختبارين).
- نسبة نجاح الفحوصات الإجمالية استقرت عند 50.00% بالتام (10 ناجحة و 10 فاشلة).
- التفسير لنسبة الـ 50%: نلاحظ أن فحص الـ Success (201) نجح بنسبة كاملة لجميع العمليات الـ 10.
- في المقابل، فحص الـ Blocked (423) فشل تماماً بنسبة 0%.
- هذا يعني أن المنظومة سمحت بمرور جميع الطلبات العشرة المتزامنة كطلبات ناجحة ومستقلة، ولم تنجح في حظر أو صد أي طلب مكرر منها.

لحل تلك المشكلة قمنا بتعديل آلية استقبال الطلبات في بداية تابع إنشاء الطلب (api/orders/create) كالتالي:

```
1 $lock = Cache::lock('order-submit-user-' . $user->id, 10);
2
3 if (!$lock->get()) {
4     return response()->json([
5         'message' => 'The previous request is currently being processed, please wait'
6     ], 423);
7 }
```


بدلاً من إغلاق النظام بالكامل، قمنا بإنشاء قفل مخصص وديناميكي يربط اسم القفل بـ `id` المستخدم الحالي

(`$user->id`) وبصلاحية زمنية تبلغ 10 ثوان، هذا يعني أن كل مستخدم يملك مساراً معزولاً، فلو ضغط المستخدم (أ) على الزر مئة مرة، سيُفقد حسابه هو فقط دون أن يتأثر المستخدم (ب) الذي يشتري في نفس اللحظة.

بمجرد دخول الطلب الأول ، يستحوذ على القفل، وعندما تصل الطلبات التسعة الأخرى المتزامنة في نفس الأجزاء من الثانية، يفشل تابع `get()` في الحصول على القفل، فيقوم السيرفر بحظرها ليعلم التطبيق أن هناك عملية معالجة جارية حالياً ويطلب من المستخدم الانتظار.

و عند نهاية التابع وضعنا فك القفل :

```
finally {  
    $lock->release();  
}
```

بمجرد انتهاء السيرفر من معالجة الطلب الأول بنجاح وتخزينه في قاعدة البيانات (والذي يستغرق أجزاءً من الثانية فقط)، يتم استدعاء أمر التحرير `release()` فوراً داخل بلوك `finally` هذا يضمن عدم حبس المستخدم لمدة 10 ثوانٍ كاملة، بل يفتح له المجال فوراً للقيام بعمليات أخرى بمجرد انتهاء معالجة طلبه الحالي بأمان.

و بتكرار الاختبار بعد التعديل نلاحظ :

```
TOTAL RESULTS  
  
checks_total.....: 20      18.103707/s  
checks_succeeded...: 50.00% 10 out of 20  
checks_failed.....: 50.00% 10 out of 20  
  
X Success (201)  
  ↳ 10% - ✓ 1 / X 9  
X Blocked (423)  
  ↳ 90% - ✓ 9 / X 1
```

● **فحص النجاح (201) Success** استقر عند 10%: وهذا يعني أن طلب واحد فقط (1 من أصل 10) هو الذي نجح في معالجة الطلب وحصل على الاستجابة 201 Created وهذا هو السلوك المطلوب لحماية المخزون والمدفوعات.

● **فحص الحظر (423) Blocked** ارتفع إلى 90%: وهذا يعني أن 9 طلبات (9 من أصل 10) تم صدها بنجاح واكتشاف تكرارها، وأعاد السيرفر لها رمز الحظر 423.

-تحقيقا للطلب العاشر لل Structured Logging قمنا بتخصيص قنوات مستقلة لكل نظام فرعي بهدف عزل الاهتمامات و ذلك من خلال إنشاء ملف log مخصص للتقارير و للاوردرات في ال config/logging :

```
- 'analytics' => [  
-     'driver' => 'daily',  
-     'path' => storage_path('logs/analytics.log'),  
-     'level' => 'info',  
-     'days' => 30,  
-     'tap' => [App\Logging\JsonFormatterTap::class],  
- ],  
- 'order_locks' => [  
-     'driver' => 'daily',  
-     'path' => storage_path('logs/order_locks.log'),  
-     'level' => 'warning',  
-     'days' => 14,  
-     'tap' => [App\Logging\JsonFormatterTap::class],  
- ],
```

-و في job انشاء التقارير نضع :

```
if (!$lock->get()) {  
    Log::channel('analytics')->warning('Analytics Job Lock Acquisition  
Failed', [  
        'reason' => 'Another worker is already generating the report',  
        'lock_key' => 'daily-sales-report'  
    ]);  
    return;  
}  
  
try {  
    $date = Carbon::yesterday()->toDateString();  
  
    Log::channel('analytics')->info('Daily Sales Analytics Job Started',  
[  
        'target_date' => $date  
    ]);  
    Log::channel('analytics')->info('Analytics Job Chunk Processed', [  
        'chunk_items_count' => $orders->count(),  
        'first_order_id' => $orders->first()->id,  
        'last_order_id' => $orders->last()->id,  
    ]);  
}
```

```

Log::channel('analytics')->info('Daily Sales Analytics Job Finished
Successfully', [
    'target_date' => $date,
    'metrics' => [
        'total_processed_orders' => $totalOrders,
        'total_sales_amount' => $totalSales,
        'average_order_value' => $averageOrderValue,
        'paid_count' => $paidOrders,
        'canceled_count' => $canceledOrders
    ]
]);
} catch (\Exception $e) {
    Log::channel('analytics')->error('Daily Sales Analytics Job Failed',
[
    'target_date' => $date ?? null,
    'error_message' => $e->getMessage(),
    'file' => $e->getFile(),
    'line' => $e->getLine()
]);
}

```

● **رصد تعارض العمليات المتزامنة (Warning):** يقوم النظام بتسجيل تحذير فوراً في حال فشل الاستحواذ على القفل (Atomic Lock)، مما يشير إلى أن خادماً (Worker) آخر يقوم بمعالجة التقرير حالياً، وبالتالي يمنع تكرار العملية وهدر الموارد.

● **توثيق مؤشر بدء التشغيل (Info):** يُسجل نقطة انطلاق المهمة بنجاح مع تحديد التاريخ المستهدف (تاريخ اليوم السابق) لضمان دقة البيانات المعالجة.

● **متابعة تقدم المعالجة الجزئية (Info):** نظراً لاعتماد آلية الحزم (Chunking) لحماية الذاكرة العشوائية، يقوم اللوغ بتوثيق نهاية معالجة كل حزمة (تضم 500 سجل) مع تسجيل المعرف الأول والأخير لها لمراقبة التقدم.

● **أرشفة المقاييس الإحصائية الناجحة (Info):** عند اكتمال المهمة بنجاح، يُصدر النظام تقريراً ختامياً شاملاً يخزن الإحصائيات المالية والعديدية النهائية المحسوبة بصيغة \$JSON\$.

● **معالجة وتحديد الأعطال الحرجة (Error):** في حال حدوث أي استثناء أو انهيار برمجى غير متوقع، يلتقطه النظام فوراً ويقوم بتوثيقه بأعلى مستوى خطورة، شاملاً (رسالة الخطأ، اسم الملف، ورقم السطر المتسبب بالعطل) لتسهيل الصيانة السريعة.

النتيجة تخزن بملف analytics-2026-06-20.log

-وللاوردرات :

```
if (!$lock->get()) {
    Log::channel('order_locks')->warning('Duplicate Order Click
Detected', [
        'user_id'    => $user->id,
        'lock_key'    => 'order-submit-user-' . $user->id,
        'ip_address' => $request->ip(),
        'url'         => $request->fullUrl(),
    ]);
    return response()->json([
        'message' => 'The previous request is currently being processed,
please wait'
    ], 423);
}
$profile['0_app_lock_acquire_ms'] = (hrtime(true) - $stepStart) / 1e6;

Log::channel('order_locks')->info('Order Lock Acquired (First Request)',
[
    'user_id' => $user->id,
    'lock_key' => 'order-submit-user-' . $user->id,
]);
```

•توثيق الاستحواذ الناجح: (Successful Lock Acquisition - Info) يقوم النظام بتسجيل حدث تشغيلي بمستوى info فور نجاح الطلب الأول (First Request) في حجز القفل الذري الحصري عبر الـ Redis مما يثبت بدء المعالجة الآمنة للطلب.

•رصد وإحباط الطلبات المتكررة: (Duplicate Click Detection - Warning) في حال ورود طلب متزامن آخر لنفس المستخدم قبل تحرير القفل، يفشل في عبور الشرط، ويقوم النظام فوراً بتسجيل تحذير بمستوى warning يوثق محاولة التكرار، مع إرفاق المعطيات الرقمية والأمنية للمحاولة (مثل: معرف المستخدم، عنوان الـ IP والرابط المستهدف)، ومن ثم تجميد الطلب وإرجاع استجابة الحظر الرقمية هيدر 423 Locked

النتيجة تخزن بملف order_locks-2026-06-20.log

المتطلب الثامن (Transaction Integrity / ACID):

في تابع إنشاء الطلب وقبل إدخال تعديلات ال Transaction، قمنا بترك الكود ينفذ استعلامات التحديث بشكل مباشر، و وضعنا في نهاية التابع استثناءً برمجياً متعمداً (throw new \Exception) هذا الاستثناء بحاكي حدوث أي عطل مفاجئ شائع في الواقع (مثل: انقطاع الاتصال ببوابة الدفع، توقف سيرفر الفواتير، أو انقطاع التيار الكهربائي عن الخادم).

```
1  $productIds = $cartItems->pluck('product_id');
2  $products = Product::whereIn('id', $productIds)
3      ->lockForUpdate()
4      ->get()
5      ->keyBy('id');
6
7  foreach ($cartItems as $item) {
8      $product = $products->get($item->product_id);
9
10     if (!$product) {
11         throw new \Exception("Product not found for id {$item->product_id}");
12     }
13
14     if ($product->stock < $item->quantity) {
15         throw new \App\Exceptions\InsufficientStockException(
16             "Requested quantity exceeds available stock",
17             $product->name
18         );
19     }
20 }
21
22 $totalPrice = $cartItems->sum(fn($i) => $i->price * $i->quantity);
23
24 $order = Order::create([
25     'user_id' => $user->id,
26     'total_price' => $totalPrice,
27 ]);
28
29 foreach ($cartItems as $item) {
30     $product = $products->get($item->product_id);
31
32     // sleep(2);
33     $product->decrement('stock', $item->quantity);
34
35     OrderItem::create([
36         'order_id' => $order->id,
37         'product_id' => $item->product_id,
38         'quantity' => $item->quantity,
39         'price' => $item->price,
40     ]);
41 }
42 throw new \Exception("unexpected error occurred");
43 // $cart->cartItems()->whereIn('id', $cartItemIds)->delete();
44 return $order;
45
46 ProcessOrder::dispatch($order, $user->id, $request->scenario);
```

و ليكن لدينا المنتج رقم 5 بالداتا بيز بمخزون 26 :

	id	name	description	price	stock
	bigint unsigned	varchar(255)	text	decimal(10,2)	int
>	5	ex	Qui velit et mollitia esse cor	110.00	26

و هنا جدول ال orders بالشكل :

	id	order_id	product_id	quantity	price	created_at
	bigint unsigned	bigint unsigned	bigint unsigned	int	decimal(10,2)	timestamp
>	1	1	1	2	132.00	2026-06-17 20:55:39
>	2	2	2	2	252.00	2026-06-17 21:01:54
>	3	3	2	2	252.00	2026-06-17 21:02:29
>	4	4	2	2	252.00	2026-06-17 21:02:29
>	5	5	2	2	252.00	2026-06-17 21:02:29
>	6	6	2	2	252.00	2026-06-17 21:02:29
>	7	7	3	2	138.00	2026-06-17 21:58:19

و أيضاً جدول ال order_items:

	id	user_id	total_price	status	processed_by	invoice_path
	bigint unsigned	bigint unsigned	decimal(10,2)	enum('pending','shipp	varchar(255)	varchar(255)
>	1	202	264.00	pending	(NULL)	(NULL)
>	2	202	504.00	pending	(NULL)	(NULL)
>	3	202	504.00	pending	(NULL)	(NULL)
>	4	202	504.00	pending	(NULL)	(NULL)
>	5	202	504.00	pending	(NULL)	(NULL)
>	6	202	504.00	pending	(NULL)	(NULL)
>	7	202	276.00	pending	(NULL)	(NULL)

و عندما قام مستخدم ب order بالشكل :

POST	http://127.0.0.1:8080/api/orders/create	Send
Params	Body	Response
	JSON	500 Internal Server Error 349ms 70B
1	{	1 {
2	"items": [5]	2 "error": "Something went wrong",
3	}	3 "details": "unexpected error occurred"
		4 }

أعطى الخطأ المقصود وبالاطلاع على الداتا بيز للمنتج رقم 5 الذي حاولنا طلبه نلاحظ :

	id	name	description	price	stock
	bigint unsigned	varchar(255)	text	decimal(10,2)	int
>	5	ex	Qui velit et mollitia esse cor	110.00	24

بالرغم من أن الاستثناء البرمجي قد انطلق، وتوقفت عملية إنشاء الطلب بالكامل ولم يتم إصدار أي فاتورة أو رقم طلب للمستخدم، إلا أننا عند فحص قاعدة البيانات وجدنا أن مخزون المنتج قد انخفض .

تفسير النتيجة :

● عند غياب المعاملات البرمجية (Transaction) فإن قاعدة البيانات تعامل كل سطر كود كعملية منفصلة تماماً وتثبتها في الجداول فوراً لذلك، عندما مر الكود على أمر إنقاص المخزون، تم التعديل فوراً في الداتابيز.

● فقدان خاصية الذرية (Atomicity) وهي الركن الأول من مبادئ ACID والتي تنص على أن مجموعة العمليات المرتبطة ببعضها يجب أن تنفذ ككتلة واحدة إما أن تنجح كلها معاً، أو تفشل كلها معاً دون ترك أي أثر All or Nothing

كما تم إنشاء order و order_item :

	id bigint unsigned	order_id bigint unsigned	product_id bigint unsigned	quantity int	price decimal(10,2)	
>	2	2	2	2	252.00	20
>	3	3	2	2	252.00	20
>	4	4	2	2	252.00	20
>	5	5	2	2	252.00	20
>	6	6	2	2	252.00	20
>	7	7	3	2	138.00	20
>	9	9	5	2	220.00	20

(id =9) للمنتج الخامس لم يكن موجود و تم إنشاؤه)

	id bigint unsigned	user_id bigint unsigned	total_price decimal(10,2)	status enum('pending','shipp pending	processed_by varchar(255)
>	2	202	504.00	pending	(NULL)
>	3	202	504.00	pending	(NULL)
>	4	202	504.00	pending	(NULL)
>	5	202	504.00	pending	(NULL)
>	6	202	504.00	pending	(NULL)
>	7	202	276.00	pending	(NULL)
>	9	202	440.00	pending	(NULL)

لحل المشكلة قمنا بإعادة هيكلة التابع بالكامل وتغليفه بالمعاملات البرمجية لضمان حماية قاعدة البيانات وضمان تنفيذ العمليات بمبدأ الكل أو لا شيء.

```

1  $order = DB::transaction(function () use ($user, $cartItems, $cart, $cartItemIds) {
2
3      $productIds = $cartItems->pluck('product_id');
4      $products = Product::whereIn('id', $productIds)
5          ->lockForUpdate()
6          ->get()
7          ->keyBy('id');
8
9      foreach ($cartItems as $item) {
10         $product = $products->get($item->product_id);
11
12         if (!$product) {
13             throw new \Exception("Product not found for id {$item->product_id}");
14         }
15
16         if ($product->stock < $item->quantity) {
17             throw new \App\Exceptions\InsufficientStockException(
18                 "Requested quantity exceeds available stock",
19                 $product->name
20             );
21         }
22     }
23
24     $totalPrice = $cartItems->sum(fn($i) => $i->price * $i->quantity);
25
26     $order = Order::create([
27         'user_id' => $user->id,
28         'total_price' => $totalPrice,
29     ]);
30
31     foreach ($cartItems as $item) {
32         $product = $products->get($item->product_id);
33
34         // sleep(2);
35         $product->decrement('stock', $item->quantity);
36
37         OrderItem::create([
38             'order_id' => $order->id,
39             'product_id' => $item->product_id,
40             'quantity' => $item->quantity,
41             'price' => $item->price,
42         ]);
43     }
44     throw new \Exception("unexpected error occurred");
45     // $cart->cartItems()->whereIn('id', $cartItemIds)->delete();
46     return $order;
47 });
48
49 ProcessOrder::dispatch($order, $user->id, $request->scenario)->afterCommit();

```

قمنا باستخدام التابع `DB::transaction()` وتميرير الكود بالكامل بداخل دالة مغلقة (Closure)

يكتب عادةً ثلاثة أوامر منفصلة ويدوية وهي `DB::beginTransaction` لبدء العملية، ثم `DB::commit` في حال النجاح لتثبيت البيانات، ثم كتابة كتلة `catch` يدوية لاستدعاء `DB::rollback` في حال الفشل

أما في طريقتنا المعتمدة بـ `DB::transaction:` لقد اخترنا هذه الطريقة لأنها أكثر أماناً واحترافية وتمنع الأخطاء البشرية حيث يقوم Laravel تلقائياً بإدارة دورة حياة المعاملة بالكامل:

- بمجرد الدخول للدالة، يتم فتح الاتصال الآمن مع قاعدة البيانات.
- في حال انطلق أي استثناء (Exception) في أي سطر داخل الدالة، يكتشف السيرفر ذلك فوراً ويقوم بعمل التراجع التلقائي (Automatic Rollback) وإلغاء كل التعديلات السابقة في الجداول.
- في حال مر الكود كاملاً بسلام دون أخطاء، يتم تنفيذ التثبيت التلقائي (Automatic Commit)

و بعد انتهاء ال transaction وضعنا السطر :

```
ProcessOrder::dispatch($order, $user->id, $request->scenario)->afterCommit();
```

تماشياً مع الممارسات الهندسية الفضلى لضمان خفة واستقرار المعاملات البرمجية، تم عزل عمليات الطوابير الخلفية (Queue Dispatching) عن مسار الترانزكشن باستخدام ميزة `afterCommit` هذا الإجراء يضمن تقليص زمن أفعال قاعدة البيانات (Database Locks) إلى الحد الأدنى، ويمنع حدوث اختناقات في الاتصال Connection Pooling (Bottlenecks) تحت ظروف الضغط العالي، كما يضمن عدم انطلاق أي مهمة خلفية لمعالجة الطلب إلا بعد التأكد التام من ثبات وتكامل بياناته داخل الجداول.

و بعد تكرار الاختبار على المنتج 6 :

The image shows two parts. The top part is a screenshot of a database table with columns: id (bigint unsigned), name (varchar(255)), description (text), price (decimal(10,2)), and stock (int). The table contains one row with id 6, name 'accusamus', description 'Qui quos et ex nihil velit po', price 31.00, and stock 34. The bottom part is a screenshot of an API client showing a POST request to 'http://127.0.0.1:8080/api/orders/create'. The request body is JSON: {"items": [6]}. The response is a 500 Internal Server Error with a message: {"error": "Something went wrong", "details": "unexpected error occurred"}.

نلاحظ أن مخزون المنتج بالبيانات بيز بقي كما هو :

id	name	description	price	stock
6	accusamus	Qui quos et ex nihil velit po	31.00	34

و لم يتم إنشاء order أو order_item نهائياً و بذلك نكون قد حققنا المطلوب.

المتطلب التاسع (Stress Testing):

في هذا الطلب قمت بتنفيذ ملفين يقومان باختبار قدرة النظام على تخديم 100 مستخدم بكفاءة

الملف الأول و اسمه stress_test.js يحوي عدّة أشياء تحاكي رحلة المستخدم بدءا من تسجيل الدخول و انتهاء ب طلب order.
رحلة المستخدم ستكون على الشكل التالي :

- 1- انشاء 100 مستخدم وهمي و تسجيل دخولهم عن طريق ال token الذي تم توليده من تابع ال login الذي قمنا باستدعائه ضمن ملف الاختبار
- 2- كل مستخدم سيقوم بالبحث عن keys معينة قمت بتحديد لها مسبقا
- 3- بعد البحث سيقوم كل مستخدم بوضع المنتج الذي رقمه 1 في الداتابيز ضمن السلة الخاصة به
- 4- بعد وضعه ضمن السلة سيقوم بانشاء أوردر وفي هذه الحالة سيحدث لدينا أمران الأول : سينجح الطلب الخاص بالمستخدم لتوفر مخزون كافي الثاني : بعد انتهاء المخزون الخاص بهذا المنتج سيتعذر انشاء طلب و ستظهر نتيجة فشل لكل المستخدمين الذين يحاولون طلب هذا المنتج بسبب out of stock

الفرق بين الملف الأول stress_test و stress_test2 أنه ضمن الملف الأول كان كل المستخدمين يطلبون نفس المنتج
اما ضمن الملف الثاني ف قمت بجعل كل مستخدم يضع عدة منتجات ضمن السلة الخاصة به و يطلبها يعني العملية ستكون على أكثر من منتج وليس منتج واحد و هذا يعتبر أكثر واقعية لرحلة المستخدم ضمن التطبيق التجاري.

بعد تطبيق الاختبارات على k6 كانت النتائج كالتالي :

كان المخزون ضمن المنتج 1 = 30. وانا لدي فعليا 100 مستخدم
عدد الطلبات التي نجحت من اصل 100 هي 30 كما يظهر في الصورة
أما ماتبقى من المستخدمين حصلوا على رسالة out of stock

يظهر أيضا في الصورة فشل في عملية الإضافة للسلة وهذا بسبب وجود شرط يتحقق من أن المنتج الذي تتم اضافته ليس موجودا في نفس الوقت.

بعد الانتهاء من الاختبار تأكدنا من المنتج أن المخزون لم يصبح 1- مثلا و بقي 0 وهذا ما يؤكد سلامة العمليات.

```

ERRO[0099] ✖ Cart Add Failed! Status: 400 source=console
ERRO[0099] ✖ Cart Add Failed! Status: 400 source=console
ERRO[0099] ✖ Cart Add Failed! Status: 400 source=console
ERRO[0099] ✖ Cart Add Failed! Status: 400 source=console
ERRO[0099] ✖ Cart Add Failed! Status: 400 source=console
ERRO[0099] ✖ Cart Add Failed! Status: 400 source=console
ERRO[0100] ✖ Cart Add Failed! Status: 400 source=console
ERRO[0100] ✖ Cart Add Failed! Status: 400 source=console

■ TOTAL RESULTS

checks_total.....: 6759 65.396812/s
checks_succeeded...: 50.58% 3419 out of 6759
checks_failed.....: 49.41% 3340 out of 6759

✓ Search Success (200)
✗ Cart Ready (Added or Already Exists)
  ↳ 2% - ✓ 79 / ✗ 3182
✗ Order Success (201)
  ↳ 37% - ✓ 30 / ✗ 49
✗ Order Failed - Insufficient Stock (400)
  ↳ 62% - ✓ 49 / ✗ 30
✗ Order Failed - Server Error (5xx)
  ↳ 0% - ✓ 0 / ✗ 79

HTTP
http_req_duration.....: avg=24.54ms min=8.95ms med=16.65ms max=273.59ms p(90)=41.29ms p(95)=61.4ms
{ expected_response:true }...: avg=24.88ms min=8.95ms med=15.27ms max=273.59ms p(90)=40.59ms p(95)=66.11ms
http_req_failed.....: 48.33% 3277 out of 6780
http_reqs.....: 6780 65.599998/s

EXECUTION
iteration_duration.....: avg=1.11s min=1.02s med=1.03s max=4.25s p(90)=1.08s p(95)=1.12s
iterations.....: 3261 31.551857/s
vus.....: 1 min=0 max=99
vus_max.....: 100 min=100 max=100

NETWORK
data_received.....: 48 MB 467 kB/s
data_sent.....: 3.5 MB 34 kB/s

running (1m43.4s), 000/100 VUs, 3261 complete and 0 interrupted iterations
default ✓ [=====] 000/100 VUs 1m20s
naya@Naya:~/projects/E-commerce$ |

```

```

> App\Models\Product::where('id', 1)->update(['stock' => 30]);

= 1

> App\Models\Product::where('id', 1)->first();

= App\Models\Product {#7652
  id: 1,
  name: "Gusikowski-Gerhold watch molestiae",
  description: "Alice had no idea how to begin.' He looked anxiously at the sides of it, and talking over its.",
  price: "4098.15",
  stock: 0,
  created_at: "2026-06-17 19:39:33",
  updated_at: "2026-06-17 19:42:22",
}

>

```

ضمن هذه الصورة أيضا يظهر لدينا انه يوجد عمليات تتم معالجتها في الخلفية تفيد في تكملة علاج الطلب مثل الدفع و تحديث المخزون

```
INFO Horizon started successfully.

2026-06-22 16:42:18 App\Jobs\ProcessOrder ..... RUNNING
2026-06-22 16:42:19 App\Jobs\ProcessOrder ..... RUNNING
2026-06-22 16:42:21 App\Jobs\ProcessOrder ..... 2s DONE
2026-06-22 16:42:21 App\Jobs\ProcessOrder ..... RUNNING
2026-06-22 16:42:22 App\Jobs\ProcessOrder ..... 346.74ms DONE
2026-06-22 16:42:22 App\Jobs\ProcessOrder ..... RUNNING
2026-06-22 16:42:21 App\Jobs\ProcessOrder ..... 2s DONE
2026-06-22 16:42:21 App\Jobs\ProcessOrder ..... RUNNING
2026-06-22 16:42:22 App\Jobs\ProcessOrder ..... 326.84ms DONE
2026-06-22 16:42:22 App\Jobs\ProcessOrder ..... RUNNING
2026-06-22 16:42:22 App\Jobs\ProcessOrder ..... 367.55ms DONE
2026-06-22 16:42:22 App\Jobs\ProcessOrder ..... RUNNING
2026-06-22 16:42:54 App\Jobs\ProcessOrder ..... 31s DONE
2026-06-22 16:42:54 App\Jobs\ProcessOrder ..... RUNNING
2026-06-22 16:42:23 App\Jobs\ProcessOrder ..... -30,984.04ms DONE
2026-06-22 16:42:23 App\Jobs\ProcessOrder ..... RUNNING
2026-06-22 16:42:22 App\Jobs\ProcessOrder ..... 367.73ms DONE
2026-06-22 16:42:22 App\Jobs\ProcessOrder ..... RUNNING
2026-06-22 16:42:54 App\Jobs\ProcessOrder ..... 31s DONE
2026-06-22 16:42:54 App\Jobs\ProcessOrder ..... RUNNING
2026-06-22 16:42:23 App\Jobs\ProcessOrder ..... -30,960.45ms DONE
2026-06-22 16:42:23 App\Jobs\ProcessOrder ..... RUNNING
2026-06-22 16:42:23 App\Jobs\ProcessOrder ..... 458.87ms DONE
2026-06-22 16:42:23 App\Jobs\ProcessOrder ..... RUNNING
2026-06-22 16:42:24 App\Jobs\ProcessOrder ..... 461.83ms DONE
2026-06-22 16:42:24 App\Jobs\ProcessOrder ..... RUNNING
2026-06-22 16:42:23 App\Jobs\ProcessOrder ..... 459.84ms DONE
2026-06-22 16:42:23 App\Jobs\ProcessOrder ..... RUNNING
2026-06-22 16:42:24 App\Jobs\ProcessOrder ..... 452.80ms DONE
2026-06-22 16:42:24 App\Jobs\ProcessOrder ..... RUNNING
2026-06-22 16:42:24 App\Jobs\ProcessOrder ..... 456.01ms DONE
```

هنا أيضا لدينا الاختبار الثاني الذي وضعت فيه أيضا تعليمات لمراقبة ما يحدث ضمن سير العمليات حيث نجد تفاصيل كل طلب مثل حالة الطلب هل هو ناجح ام فاشل وأسباب فشله وتفاصيل أخرى هذه التفاصيل تفيد المتطلب العاشر والذي يطلب مراقبة النظام بشكل حي.

طبعا كل شيء يتم طباعته اثناء عملية تشغيل ملف الاختبار يتم تخزينه ضمن ملف Log وسنحدث عن هذا بالتفصيل في الطلب الأخير.


```

INFO[0041] [VU-67-ITER-4] RESPONSE STATUS: 400 source=console
INFO[0041] [VU-77-ITER-4] SUCCESS | User 77 bought: [P16:Q2, P12:Q2, P19:Q2] source=console
INFO[0041] [VU-77-ITER-4] RESPONSE STATUS: 201 source=console
INFO[0041] [VU-62-ITER-4] OUT OF STOCK | User 62 tried to buy: [P11:Q1, P3:Q1] but failed. Item: Schuster Ltd bag optio source=console
INFO[0041] [VU-62-ITER-4] RESPONSE STATUS: 400 source=console
INFO[0041] [VU-21-ITER-4] SUCCESS | User 21 bought: [P20:Q2, P6:Q1] source=console
INFO[0041] [VU-21-ITER-4] RESPONSE STATUS: 201 source=console
INFO[0041] [VU-9-ITER-4] SUCCESS | User 9 bought: [P20:Q1] source=console
INFO[0041] [VU-9-ITER-4] RESPONSE STATUS: 201 source=console
INFO[0041] [VU-44-ITER-4] OUT OF STOCK | User 44 tried to buy: [P6:Q1, P11:Q2, P3:Q1] but failed. Item: Schuster Ltd bag optio source=console
INFO[0041] [VU-44-ITER-4] RESPONSE STATUS: 400 source=console
INFO[0041] [VU-26-ITER-4] OUT OF STOCK | User 26 tried to buy: [P8:Q2, P10:Q2, P9:Q2, P14:Q1, P20:Q1, P2:Q1, P16:Q1, P4:Q2, P6:Q1, P19:Q2] but failed. Item: Romaguera PLC camera architecto source=console
INFO[0041] [VU-26-ITER-4] RESPONSE STATUS: 400 source=console
INFO[0041] [VU-20-ITER-4] SUCCESS | User 20 bought: [P16:Q2, P6:Q1] source=console
INFO[0041] [VU-20-ITER-4] RESPONSE STATUS: 201 source=console
INFO[0041] [VU-74-ITER-4] OUT OF STOCK | User 74 tried to buy: [P17:Q2, P1:Q2, P13:Q1, P15:Q1, P12:Q2, P19:Q2, P20:Q2, P4:Q2] but failed. Item: Langosh Inc shoes eum source=console
INFO[0041] [VU-74-ITER-4] RESPONSE STATUS: 400 source=console
INFO[0041] [VU-78-ITER-4] SUCCESS | User 78 bought: [P17:Q1, P2:Q1] source=console
INFO[0041] [VU-78-ITER-4] RESPONSE STATUS: 201 source=console
INFO[0041] [VU-85-ITER-4] SUCCESS | User 85 bought: [P19:Q2, P4:Q2] source=console
INFO[0041] [VU-85-ITER-4] RESPONSE STATUS: 201 source=console
INFO[0041] [VU-17-ITER-4] OUT OF STOCK | User 17 tried to buy: [P20:Q1, P7:Q1, P9:Q2, P13:Q2, P17:Q2, P16:Q1, P6:Q2] but failed. Item: Goldner, Turner and Kertzmann samsung est source=console
INFO[0041] [VU-17-ITER-4] RESPONSE STATUS: 400 source=console
INFO[0041] [VU-5-ITER-4] SUCCESS | User 5 bought: [P2:Q2, P16:Q2, P3:Q2] source=console
INFO[0041] [VU-5-ITER-4] RESPONSE STATUS: 201 source=console
INFO[0041] [VU-79-ITER-4] OUT OF STOCK | User 79 tried to buy: [P1:Q1, P9:Q1, P12:Q1, P13:Q1, P3:Q2, P16:Q2] but failed. Item: Langosh Inc shoes eum source=console
INFO[0041] [VU-79-ITER-4] RESPONSE STATUS: 400 source=console
INFO[0041] [VU-86-ITER-4] SUCCESS | User 86 bought: [P16:Q1, P17:Q2, P19:Q1] source=console
INFO[0041] [VU-86-ITER-4] RESPONSE STATUS: 201 source=console

```

TOTAL RESULTS

```

checks_total.....: 2713 63.77141/s
checks_succeeded...: 67.78% 1839 out of 2713
checks_failed.....: 32.21% 874 out of 2713

```

```

✓ Search Success (200)
X Cart Item Added/Exists/Out of Stock
  ↳ 91% - ✓ 1003 / X 99
X Order Success (201)
  ↳ 71% - ✓ 276 / X 109
X Logical Failure - Out of Stock/Validation (400)
  ↳ 27% - ✓ 104 / X 281
X Technical Failure - Server Crash (5xx)
  ↳ 0% - ✓ 0 / X 385

```

CUSTOM

```

errors_logical_out_of_stock....: 104 2.44461/s
mysql_lock_wait_time_ms.....: avg=329.654628 min=0.982406 med=95.433078 max=2514.046613 p(90)=984.086946 p(95)=1260.050843
php_processing_time_ms.....: avg=520.887904 min=9.62933 med=327.273205 max=2596.061853 p(90)=1238.131258 p(95)=1598.468511

```

HTTP

```

http_req_duration.....: avg=3.51s min=91.31µs med=478.64ms max=34.16s p(90)=2.96s p(95)=32.44s
{ expected_response:true }...: avg=3.72s min=91.31µs med=562.83ms max=34.16s p(90)=3.14s p(95)=32.59s
http_req_failed.....: 32.31% 955 out of 2955
http_reqs.....: 2955 69.459829/s

```

EXECUTION

```

iteration_duration.....: avg=7.03s min=4.04s med=6.95s max=11.2s p(90)=9.51s p(95)=10.03s
iterations.....: 456 10.718674/s
vus.....: 24 min=0 max=100
vus_max.....: 100 min=100 max=100

```

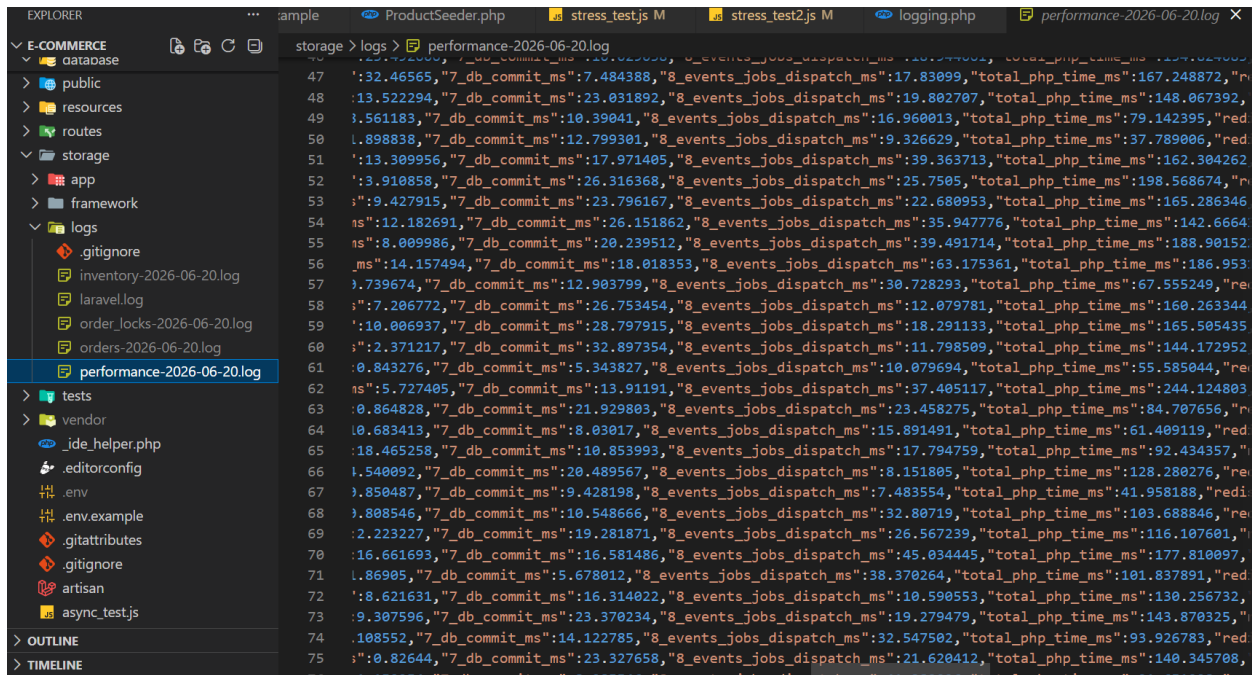
المتطلب العاشر والأخير (Benchmarking and Bottleneck) :(Analysis)

لضمان قدرة النظام على التعامل مع ال High Concurrency و الضغط الشديد دون انهيار قمنا ببناء معمارية تعتمد على قياس زمن التنفيذ و توجيه السجلات Structured Logging

لتحقيق شرط ال Benchmarking وتحليل الاختناقات، اتبعنا المنهجية الهندسية التالية:

- ١ - القياس المنهجي لزمن تنفيذ العمليات (Micro-Profiling)
لمعرفة مكان ابطاء الطلب بالتحديد قمنا بوضع مؤشرات قياس الأداء داخل الكود الخاص بإنشاء الطلب باستخدام الدالة hrtime(true) حيث انها تقيس زمن تنفيذ الطلب بدقة
- بالإضافة الى أنه تم تقسيم دورة حياة الطلب إلى أجزاء دقيقة(Micro-steps) ، وحساب الزمن المستغرق لكل خطوة بال-Millisecond
- ٢ - نظام السجلات المهيكلية(Structured Logging Architecture)
لإدارة حجم البيانات الهائل الناتج عن آلاف الطلبات المتزامنة، قمنا بالتخلي عن ملف السجلات الافتراضي الموحد، وبنينا قنوات (Channels) منفصلة ومتخصصة لتسهيل عملية ال Debugging واكتشاف الأخطاء الجذرية. تركزت تحليلاتنا على ملفين أساسيين:

أ. سجلات الأداء (performance.log): تم تخصيص هذا الملف حصرياً لتسجيل المقاييس الزمنية (Metrics) لكل طلب ناجح. من خلال هذا الملف، استطعنا مراقبة مؤشرات الأداء الحيوية واكتشاف أي ارتفاع مفاجئ (Spike) في زمن الاستجابة، مما وفر لنا رؤية واضحة حول كفاءة الخادم تحت الضغط.



```
47 :32.46565,"7_db_commit_ms":7.484388,"8_events_jobs_dispatch_ms":17.83099,"total_php_time_ms":167.248872,"r
48 :13.522294,"7_db_commit_ms":23.031892,"8_events_jobs_dispatch_ms":19.802707,"total_php_time_ms":148.067392,
49 :561183,"7_db_commit_ms":10.39041,"8_events_jobs_dispatch_ms":16.960013,"total_php_time_ms":79.142395,"red
50 :1.898838,"7_db_commit_ms":12.799301,"8_events_jobs_dispatch_ms":9.326629,"total_php_time_ms":37.789006,"red
51 :13.309956,"7_db_commit_ms":17.971405,"8_events_jobs_dispatch_ms":39.363713,"total_php_time_ms":162.304262
52 :3.910858,"7_db_commit_ms":26.316368,"8_events_jobs_dispatch_ms":25.7505,"total_php_time_ms":198.568674,"r
53 :9.427915,"7_db_commit_ms":23.796167,"8_events_jobs_dispatch_ms":22.680953,"total_php_time_ms":165.286346
54 :12.182691,"7_db_commit_ms":26.151862,"8_events_jobs_dispatch_ms":35.947776,"total_php_time_ms":142.6664
55 :8.009986,"7_db_commit_ms":20.239512,"8_events_jobs_dispatch_ms":39.491714,"total_php_time_ms":188.90152
56 :14.157494,"7_db_commit_ms":18.018353,"8_events_jobs_dispatch_ms":63.175361,"total_php_time_ms":186.953
57 :7.739674,"7_db_commit_ms":12.903799,"8_events_jobs_dispatch_ms":30.728293,"total_php_time_ms":67.555249,"re
58 :7.206772,"7_db_commit_ms":26.753454,"8_events_jobs_dispatch_ms":12.079781,"total_php_time_ms":160.263344
59 :10.006937,"7_db_commit_ms":28.797915,"8_events_jobs_dispatch_ms":18.291133,"total_php_time_ms":165.505435
60 :2.371217,"7_db_commit_ms":32.897354,"8_events_jobs_dispatch_ms":11.798509,"total_php_time_ms":144.172952
61 :0.843276,"7_db_commit_ms":5.343827,"8_events_jobs_dispatch_ms":10.079694,"total_php_time_ms":55.585044,"re
62 :5.727405,"7_db_commit_ms":13.91191,"8_events_jobs_dispatch_ms":37.405117,"total_php_time_ms":244.124803
63 :0.864828,"7_db_commit_ms":21.929803,"8_events_jobs_dispatch_ms":23.458275,"total_php_time_ms":84.707656,"r
64 :0.683413,"7_db_commit_ms":8.03017,"8_events_jobs_dispatch_ms":15.891491,"total_php_time_ms":61.409119,"red
65 :18.465258,"7_db_commit_ms":10.853993,"8_events_jobs_dispatch_ms":17.794759,"total_php_time_ms":92.434357,"i
66 :1.540092,"7_db_commit_ms":20.489567,"8_events_jobs_dispatch_ms":8.151805,"total_php_time_ms":128.280276,"re
67 :8.50487,"7_db_commit_ms":9.428198,"8_events_jobs_dispatch_ms":7.483554,"total_php_time_ms":41.958188,"redi
68 :8.088546,"7_db_commit_ms":10.548666,"8_events_jobs_dispatch_ms":32.80719,"total_php_time_ms":103.688846,"re
69 :2.223227,"7_db_commit_ms":19.281871,"8_events_jobs_dispatch_ms":26.567239,"total_php_time_ms":116.107601,"i
70 :16.661693,"7_db_commit_ms":16.581486,"8_events_jobs_dispatch_ms":45.034445,"total_php_time_ms":177.810097,
71 :1.86905,"7_db_commit_ms":5.678012,"8_events_jobs_dispatch_ms":38.370264,"total_php_time_ms":101.837891,"red
72 :8.621631,"7_db_commit_ms":16.314022,"8_events_jobs_dispatch_ms":10.590553,"total_php_time_ms":130.256732,
73 :9.307596,"7_db_commit_ms":23.370234,"8_events_jobs_dispatch_ms":19.279479,"total_php_time_ms":143.870325,"i
74 :10.8552,"7_db_commit_ms":14.122785,"8_events_jobs_dispatch_ms":32.547502,"total_php_time_ms":93.926783,"red
75 :0.82644,"7_db_commit_ms":23.327658,"8_events_jobs_dispatch_ms":21.620412,"total_php_time_ms":140.345708,
```

ب. سجلات المخزون والتدقيق (inventory.log):

تم تخصيص هذا الملف لمراقبة النزمانية وحماية البيانات. يقوم هذا السجل بتوثيق حركات المخزون (Stock Movements) بدقة، والأهم من ذلك، يوثق أسباب الفشل المنطقي عند محاولة شراء منتج نافذ، مع ذكر معرف التتبع (trace_id) واسم المنتج المتسبب بالفرض، مما يضمن عدم ضياع أي بيانات تتعلق بنفاذ المخزون تحت الضغط.

```
File Edit Selection View Go Run ...
inventory-2026-06-22.log
storage > logs > inventory-2026-06-22.log
1 {"message": "Stock Movement", "context": {"trace_id": "f9f2ed46-1d24-42f7-9aea-ac0549787fbb", "order_id": 1, "user_id": 58, "product_id": 1, "name": "Maggio"},
2 {"message": "Stock Movement", "context": {"trace_id": "edc02f09-a22f-4593-99e7-e109d087deeb", "order_id": 2, "user_id": 1, "product_id": 1, "name": "Maggio"},
3 {"message": "Stock Movement", "context": {"trace_id": "8b458b61-6171-4422-ac4a-cc8ad100733", "order_id": 3, "user_id": 61, "product_id": 1, "name": "Maggio"},
4 {"message": "Stock Movement", "context": {"trace_id": "c54ab470-ade0-4f28-a46a-b061250aa9e1", "order_id": 4, "user_id": 55, "product_id": 1, "name": "Maggio"},
5 {"message": "Stock Movement", "context": {"trace_id": "670a31f3-9a24-4fb1-81c9-16ef76cf30b9", "order_id": 5, "user_id": 7, "product_id": 1, "name": "Maggio"},
6 {"message": "Stock Movement", "context": {"trace_id": "4e97aae6-ddc-4709-8aec-0cb75ae91fce", "order_id": 6, "user_id": 8, "product_id": 1, "name": "Maggio"},
7 {"message": "Stock Movement", "context": {"trace_id": "608d9ee8-47d1-4737-9c99-a546a1378ad8", "order_id": 7, "user_id": 6, "product_id": 1, "name": "Maggio"},
8 {"message": "Stock Movement", "context": {"trace_id": "f9197630-42e5-4431-bca5-5d5ab26fea2a", "order_id": 8, "user_id": 99, "product_id": 1, "name": "Maggio"},
9 {"message": "Stock Movement", "context": {"trace_id": "671fcbb9-f61b-416b-a384-c4b5b0dd0b90", "order_id": 9, "user_id": 13, "product_id": 1, "name": "Maggio"},
10 {"message": "Stock Movement", "context": {"trace_id": "dff9adbb-92b8-4885-a675-f014a484689b", "order_id": 10, "user_id": 85, "product_id": 1, "name": "Maggio"},
11 {"message": "Stock Movement", "context": {"trace_id": "cc629850-8707-4b72-bf10-df7732995f2f", "order_id": 11, "user_id": 57, "product_id": 1, "name": "Maggio"},
12 {"message": "Stock Movement", "context": {"trace_id": "c5b23a02-40b1-4f99-b70d-c288c4abb1f3", "order_id": 12, "user_id": 60, "product_id": 1, "name": "Maggio"},
13 {"message": "Stock Movement", "context": {"trace_id": "eec4603d-a21f-4f92-b1da-b36f793e4468", "order_id": 13, "user_id": 10, "product_id": 1, "name": "Maggio"},
14 {"message": "Stock Movement", "context": {"trace_id": "1613de68-e8c7-4428-99c1-49ca9afd8d07", "order_id": 14, "user_id": 52, "product_id": 1, "name": "Maggio"},
15 {"message": "Stock Movement", "context": {"trace_id": "ee0bc568-76bd-4494-b748-cd5411587467", "order_id": 15, "user_id": 12, "product_id": 1, "name": "Maggio"},
16 {"message": "Stock Movement", "context": {"trace_id": "5dccc2ba-9b69-478f-9fa5-a7efb79e12c5", "order_id": 16, "user_id": 4, "product_id": 1, "name": "Maggio"},
17 {"message": "Stock Movement", "context": {"trace_id": "b8fe7046-867a-48a2-9f86-6995538a1d90", "order_id": 17, "user_id": 87, "product_id": 1, "name": "Maggio"},
18 {"message": "Stock Movement", "context": {"trace_id": "91c9746c-4129-461e-a579-f21234203de9", "order_id": 18, "user_id": 70, "product_id": 1, "name": "Maggio"},
19 {"message": "Stock Movement", "context": {"trace_id": "b2c6950e-9e8b-4155-8124-1ade0550a2de", "order_id": 19, "user_id": 54, "product_id": 1, "name": "Maggio"},
20 {"message": "Stock Movement", "context": {"trace_id": "bf2003f6-d3ba-4616-ad0e-227d111ac4a3", "order_id": 20, "user_id": 75, "product_id": 1, "name": "Maggio"},
21 {"message": "Stock Movement", "context": {"trace_id": "0a5c201b-d3ea-4a84-bd4e-06320d38144f", "order_id": 21, "user_id": 56, "product_id": 1, "name": "Maggio"},
22 {"message": "Stock Movement", "context": {"trace_id": "728a64c5-5ee8-4091-af8c-3fbedc6e45f4", "order_id": 22, "user_id": 67, "product_id": 1, "name": "Maggio"},
23 {"message": "Stock Movement", "context": {"trace_id": "30400b12-b64d-459c-92e2-41a277abb0a", "order_id": 23, "user_id": 11, "product_id": 1, "name": "Maggio"},
24 {"message": "Stock Movement", "context": {"trace_id": "5324c9d9-d31c-4341-a39e-f808aae0fb7", "order_id": 24, "user_id": 72, "product_id": 1, "name": "Maggio"},
25 {"message": "Stock Movement", "context": {"trace_id": "130730ab-bf6c-444f-bd16-3ae0fc507103", "order_id": 25, "user_id": 93, "product_id": 1, "name": "Maggio"},
26 {"message": "Stock Movement", "context": {"trace_id": "aad6ea3d-f39c-4b0b-b9c8-7f952efd45ae", "order_id": 26, "user_id": 97, "product_id": 1, "name": "Maggio"},
27 {"message": "Stock Movement", "context": {"trace_id": "6bef6b7a-4a37-4aa7-a807-82c71b5556e8", "order_id": 27, "user_id": 95, "product_id": 1, "name": "Maggio"},
28 {"message": "Stock Movement", "context": {"trace_id": "ab4b2285-b932-4aa6-81a5-7fdb9e19eb7a5", "order_id": 28, "user_id": 89, "product_id": 1, "name": "Maggio"},
29 {"message": "Stock Movement", "context": {"trace_id": "a82938e1-e21b-424c-a914-7fe4f1242c72", "order_id": 29, "user_id": 16, "product_id": 1, "name": "Maggio"},
30 {"message": "Stock Movement", "context": {"trace_id": "e9f4997b-d570-418d-927c-ab9a32181ad2", "order_id": 30, "user_id": 96, "product_id": 1, "name": "Maggio"},
31 {"message": "Stock Movement", "context": {"trace_id": "4a5c06e2-d3ba-47df-bb94-4709ca63e65e", "order_id": 31, "user_id": 59, "product_id": 1, "name": "Maggio"},
32 {"message": "Stock Movement", "context": {"trace_id": "04cd53a-12ad-48fd-86c3-7837208d9e6e", "order_id": 32, "user_id": 5, "product_id": 1, "name": "Maggio"}}
```

٣- اختبار الإجهاد التفاعلي والتتبع المباشر (Interactive Stress Testing with k6):

لم يقتصر اختبار الإجهاد (Stress Test) على إرسال طلبات وهمية (Dummy Traffic) بأعداد كبيرة، بل قمنا بكتابة سكريبت k6 متقدم يقوم بمحاكاة سلوك مستخدمين حقيقيين. (Agentic Scenario) شمل السكريبت التقنيات التالية:

تم توليد X-Trace-ID فريد لكل محاولة شراء وإرساله في الـ Header مما سمح لنا بربط كل طلب في طرفية الاختبار (Terminal) بما يقابله في ملفات الـ Logs داخل الخادم.
قام السكريبت بتصنيف النتائج فوراً وإصدار تقرير يفرق بوضوح بين:
النجاح التام (201)، الفشل المنطقي (400)، والفشل التقني (5xx)
توضح الصورة في الطلب السابق هذا الأمر.

اكتشاف الاختناق والحل المعماري (Bottleneck Discovery & Resolution)

الاستعلامات المتزامنة على قاعدة بيانات MySQL وتحديد قفل الصفوف lockForUpdate كانت هي نقطة الاختناق الرئيسية عند ارتفاع عدد المستخدمين المتزامنين، ارتفع زمن انتظار القفل بشكل حاد فأصبحت خوادم php-fpm تعاني من اختناق.

الحل كان بنقل العبء التنافسي لطبقة Redis باستخدام نصوص Lua الذرية وتحويل المهام الثقيلة لمعالجة الدفع وإنشاء الفواتير إلى طوابير المعالجة الخلفية. هذا هو الاقتراح الهندسي الذي كان سينتج عنه سرعة هائلة في الأداء ورفع نسبة استقرار النظام.

طبعاً وجدنا أنّ لهذه الطريقة (وبالرغم من سرعتها الكبيرة) سلبيات عدّة ، من أبرزها :

- ١ - الخطر الأكبر هنا هو التزامن، ففي حال نجحنا من خصم المخزون ضمن Redis ولكن أثناء محاولة تسجيل الطلب الفعلي ضمن MySQL انهار الخادم سيحدث لدينا تسرّب للمخزون. ومن ثمّ خسارة بيع منتجات حقيقية .
- ٢ - ضريبة التعقيد البرمجي ونمط الـ (Saga Pattern & Compensating Transactions Overhead) : Saga لمعالجة المشكلة المذكورة أعلاه، لم يعد بالإمكان الاعتماد على تعليمة `DB::rollback()` البسيطة. أُجبرنا معمارياً على تطبيق نمط **Saga**، وهو ما يعني كتابة شيفرات برمجية إضافية ومعقدة للتعويض والتراجع (Compensating Transactions). أي عملية تفشل في MySQL أو عند بوابة الدفع (Stripe) تتطلب إطلاق عملية معاكسة في Redis وهذا كله يرفع من تكلفة صيانة الكود.
- ٣ - مشاكل في تسخين الذاكرة: مشاكل في تسخين الذاكرة: الاعتماد على Redis كبوابة رئيسية للمخزون يعني أن النظام لا يمكنه العمل دونه. في حال انهيار سيرفر Redis وإعادة تشغيله وهو فارغ (Cold Start) ، سيفشل النظام في تلبية أي طلب شراء.

سأرفق هنا بعض الصور التي توضح أيضاً ماذا فعلنا لتحقيق متطلبات الطلب العاشر:

```
Log::channel('inventory')->warning('Stock Reservation Failed (Out of Stock)', [
    'trace_id'    => $traceId,
    'user_id'     => $user->id,
    'product_name' => $e->getProductName(),
]);
```

```
Log::channel('performance')->info("Order Profiling [#{ $order->id}]", [
    'trace_id' => $traceId,
    'metrics'  => $profile,
]);
```

هذه بعض الأماكن التي قمنا فيها بوضع تعليمات ال Log .

----- انتهى -----